

From Sensors to Actuators

Vladimir Toporkov

June 9, 2026



System Overview



System Overview



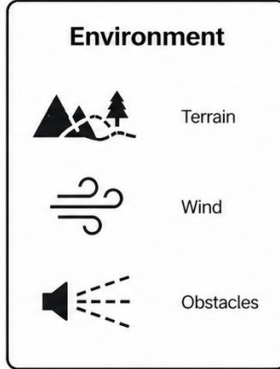
System Overview



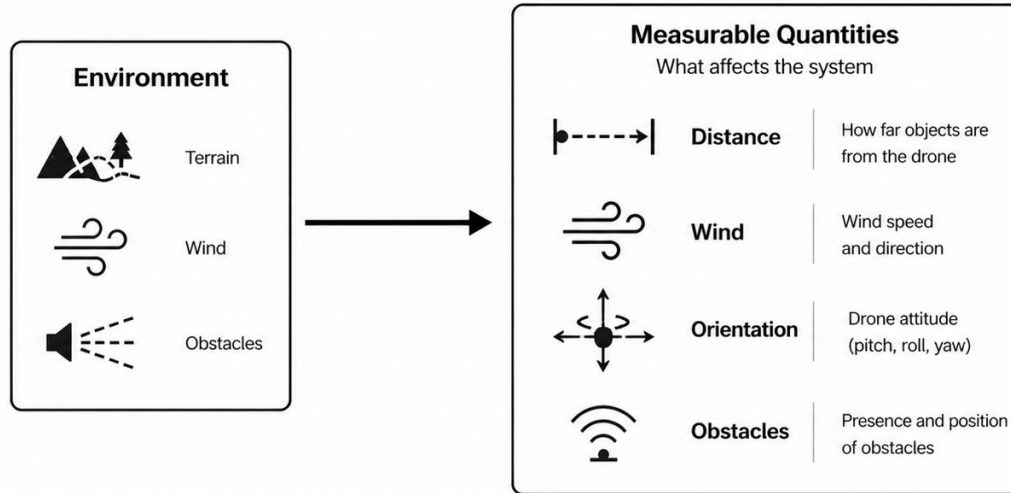
System Overview



Environment



Environment

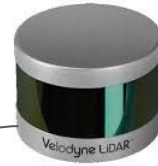


From Quantities to Sensors

Measured quantity	Sensor
--------------------------	---------------

From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic



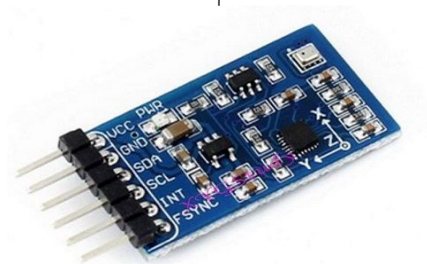
From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer



From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU



From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

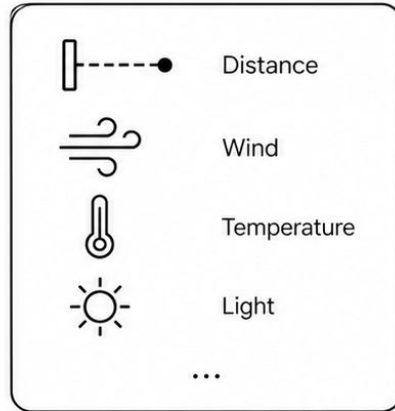


From Quantities to Sensors

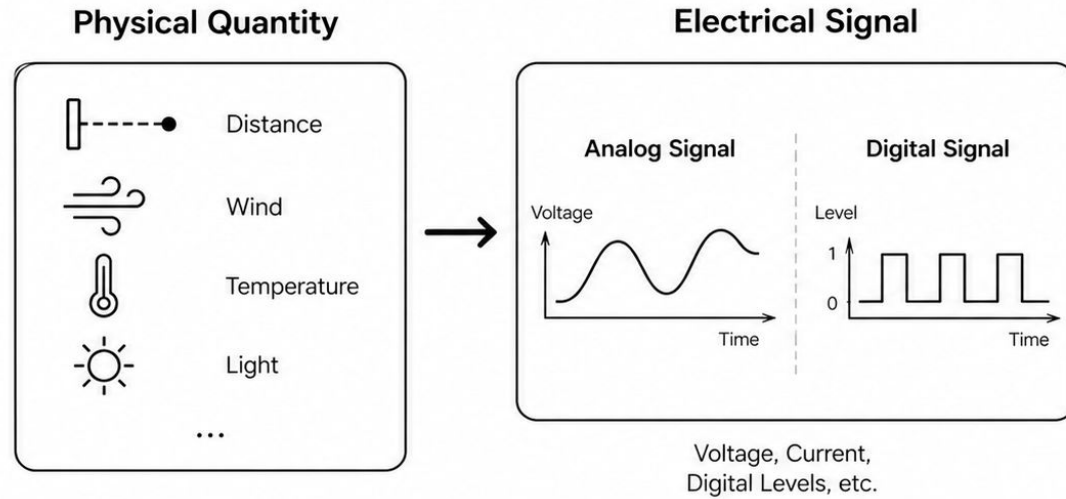
Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

What is a sensor?

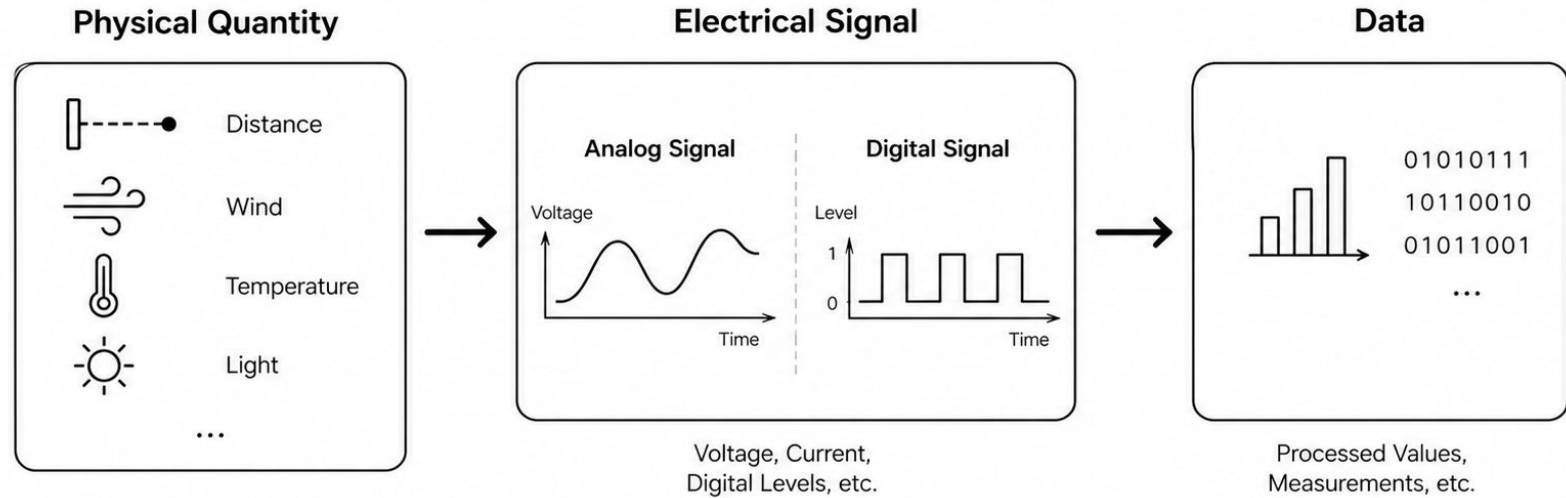
Physical Quantity



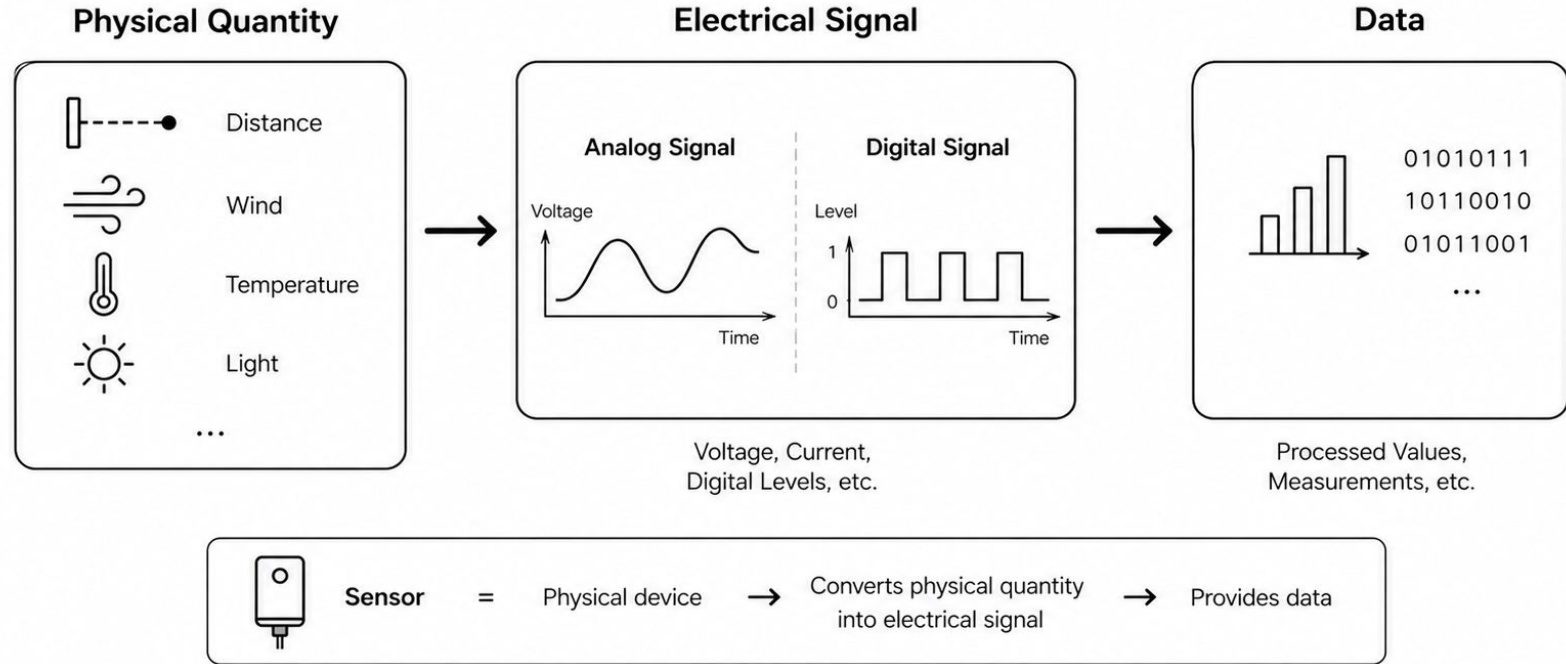
What is a sensor?



What is a sensor?



What is a sensor?



From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

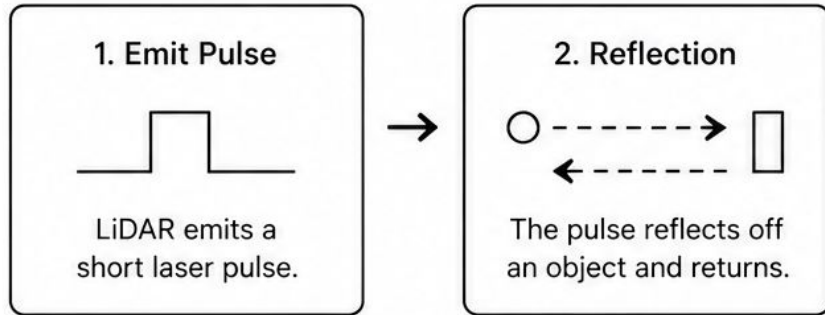
Lidar

1. Emit Pulse

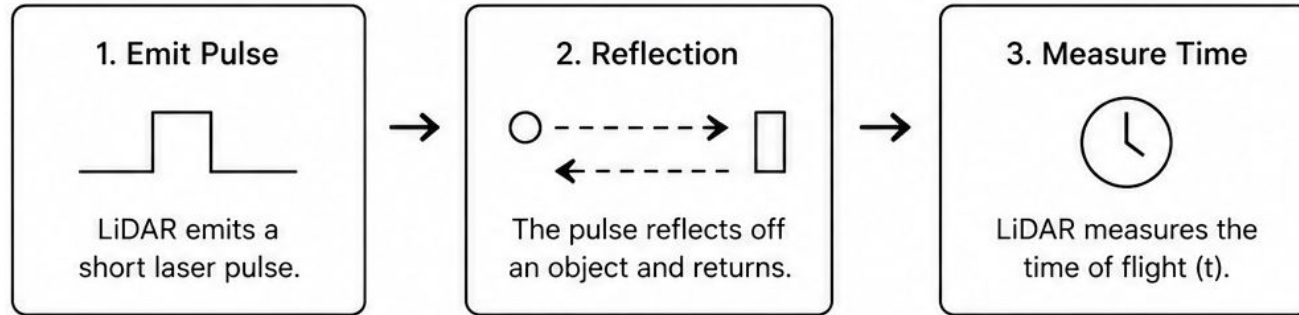


LiDAR emits a short laser pulse.

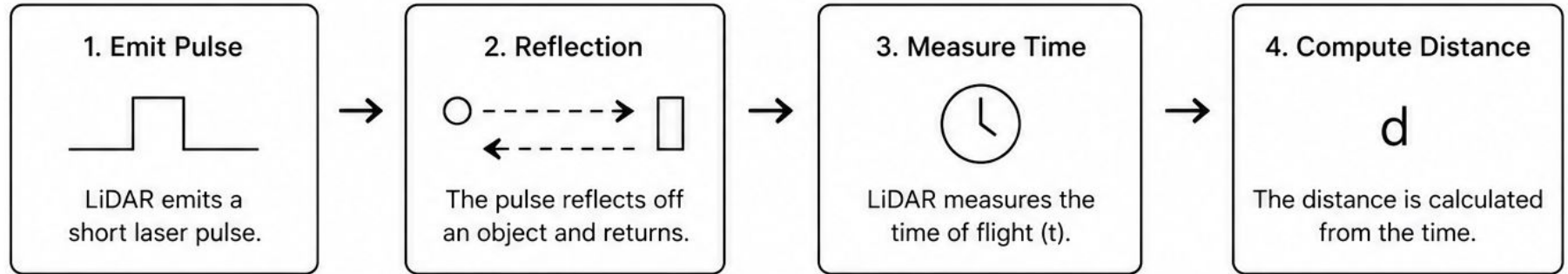
Lidar



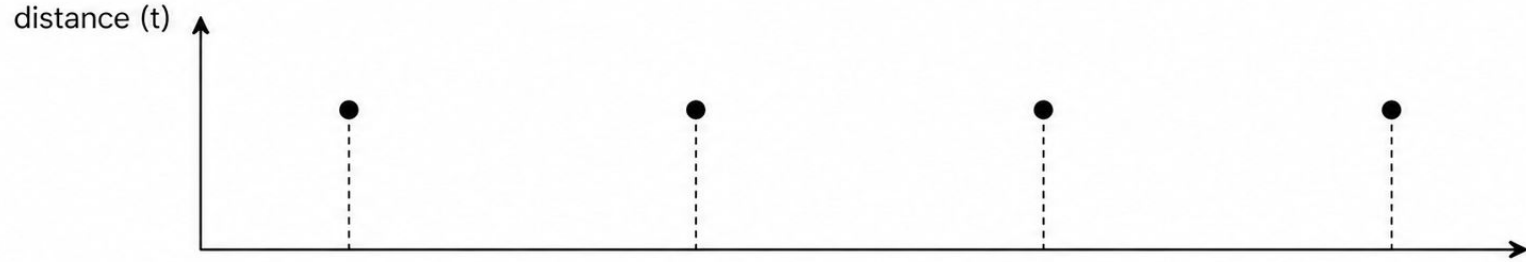
Lidar



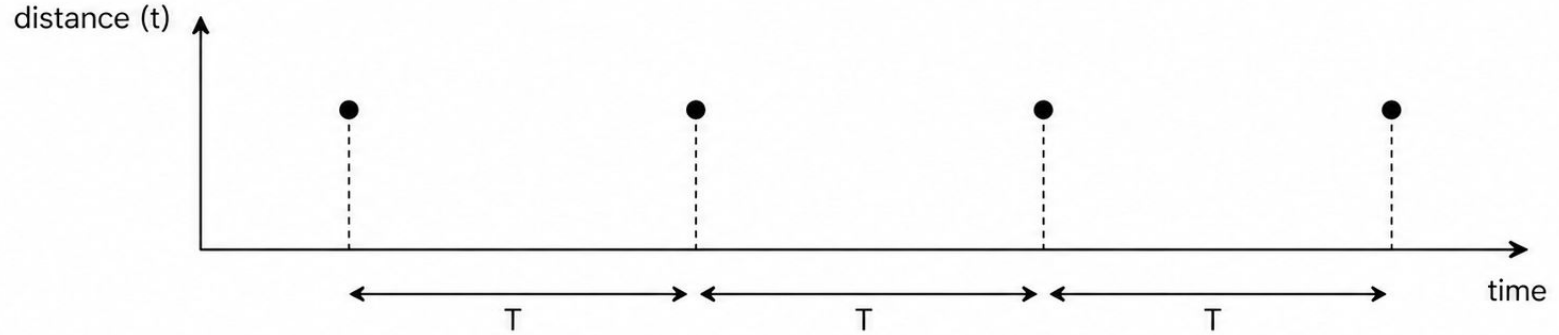
Lidar



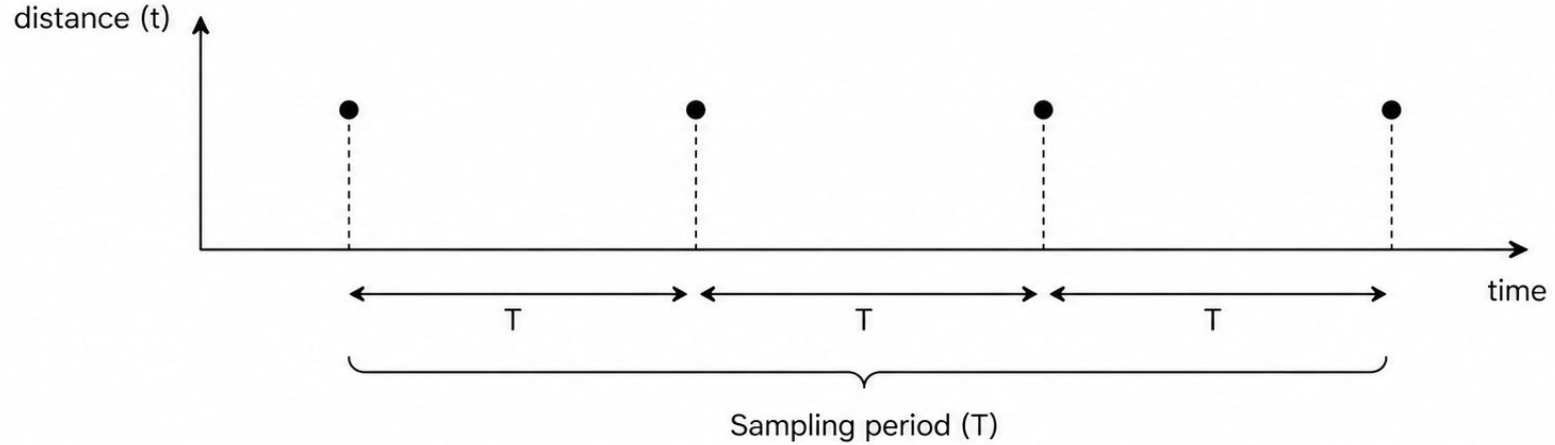
Sensor Output Over Time



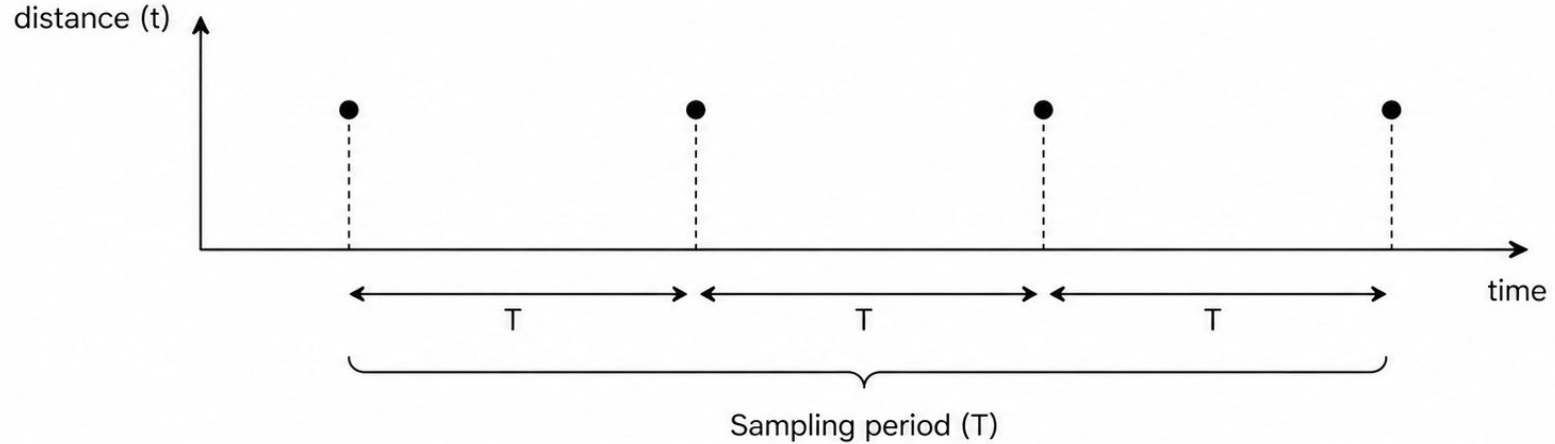
Sensor Output Over Time



Sensor Output Over Time



Sensor Output Over Time

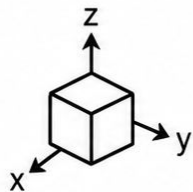


$$f = \frac{1}{T} \quad (\text{sampling frequency})$$

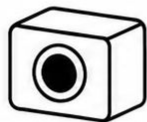
Multiple Sensors



LiDAR



IMU

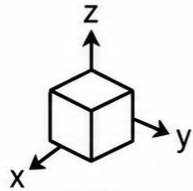
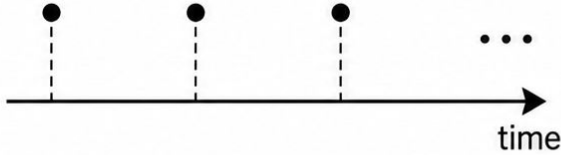


Camera

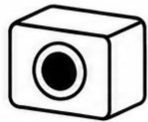
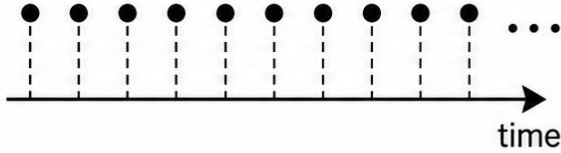
Multiple Sensors



LiDAR



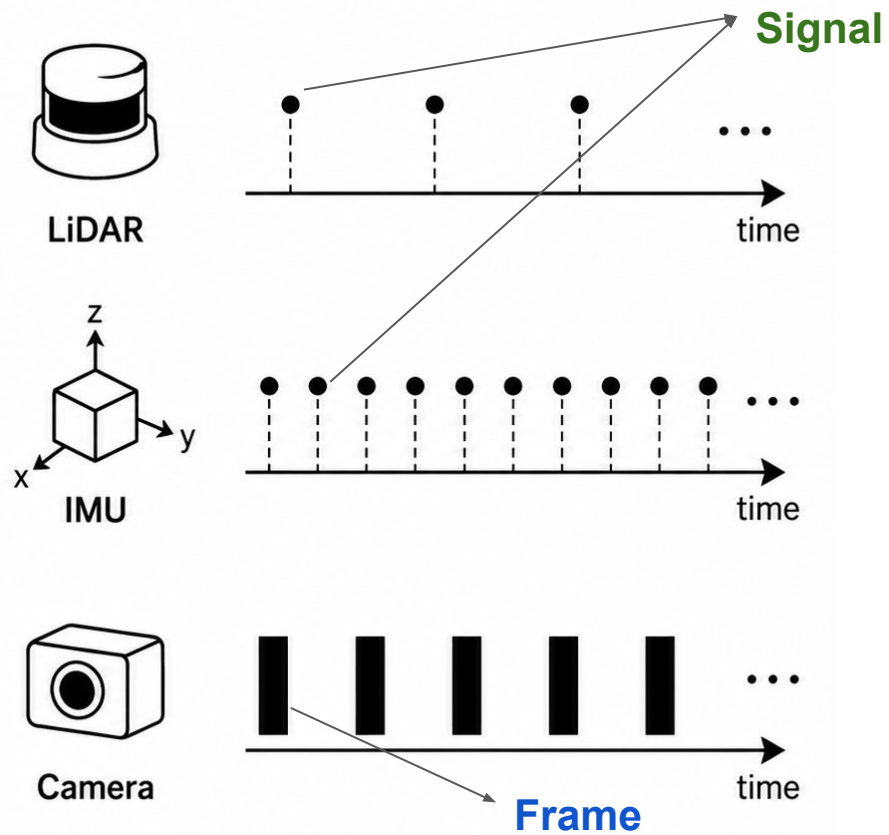
IMU



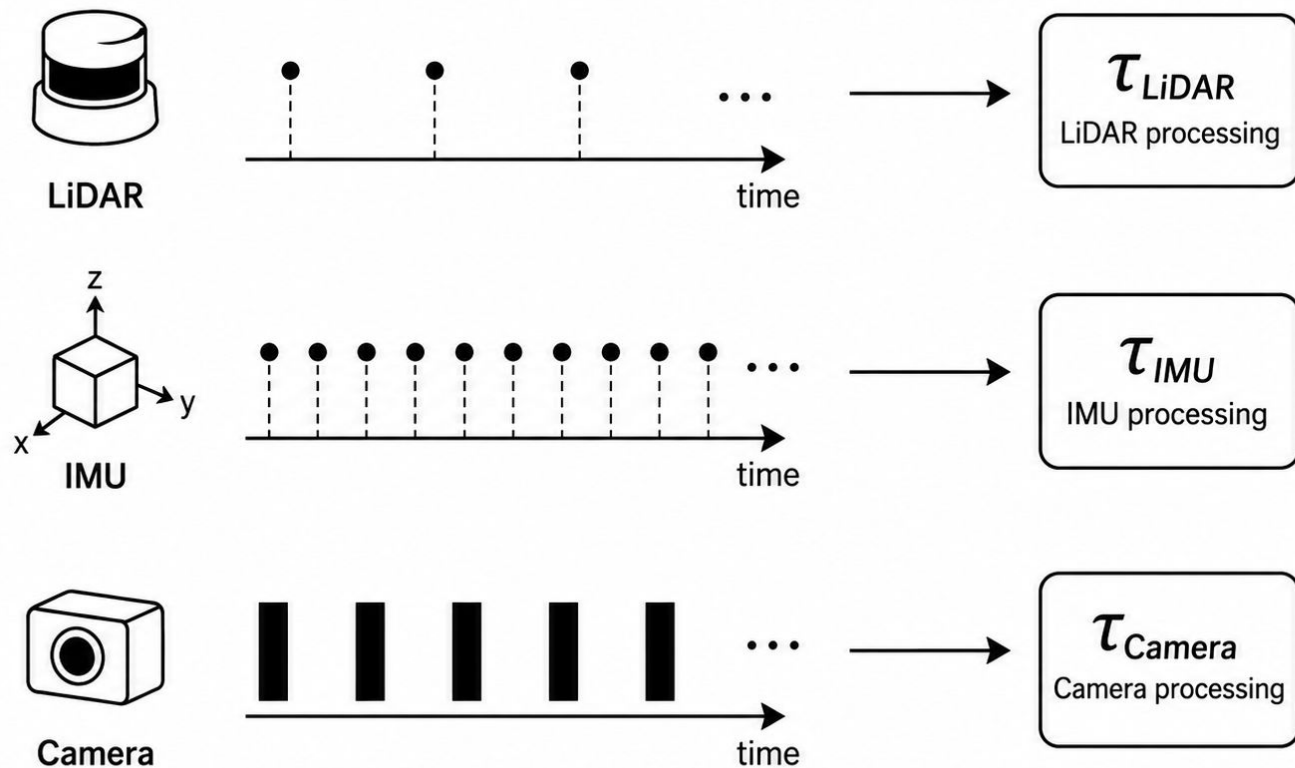
Camera



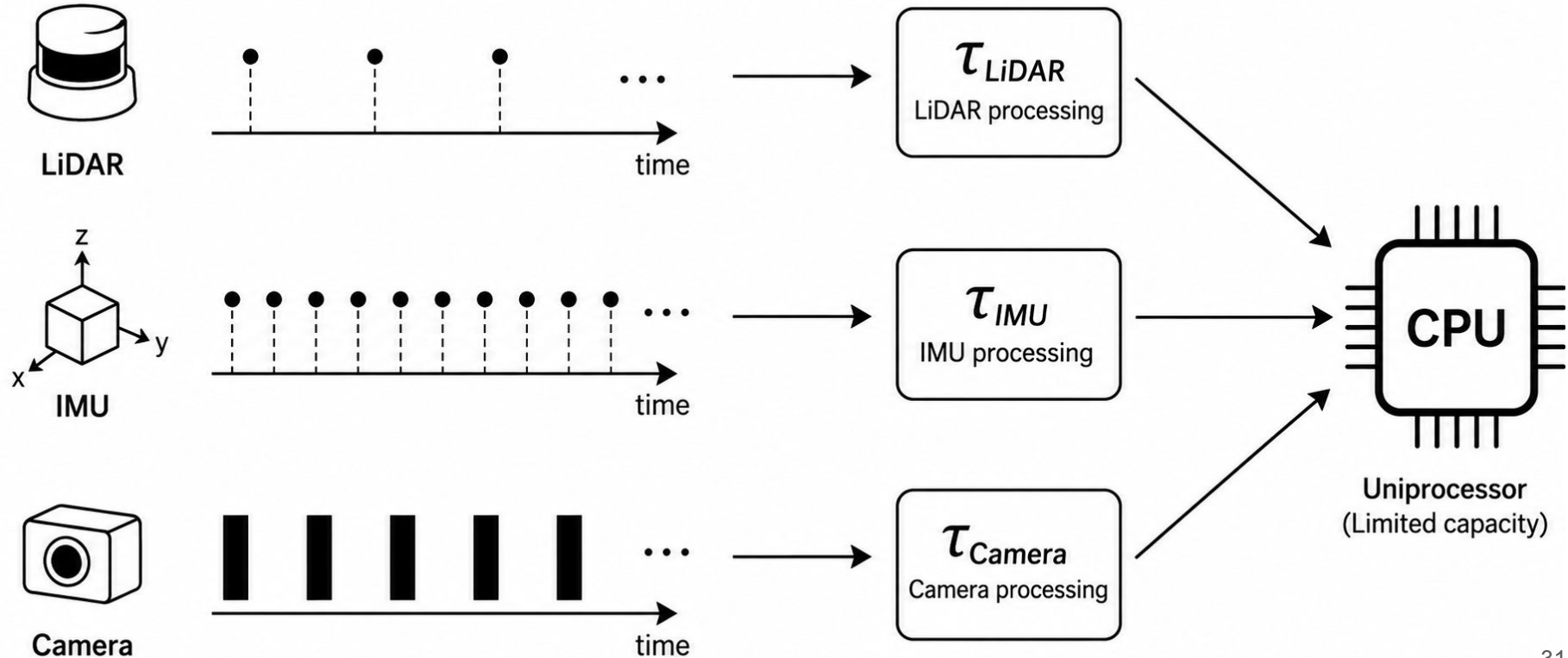
Multiple Sensors



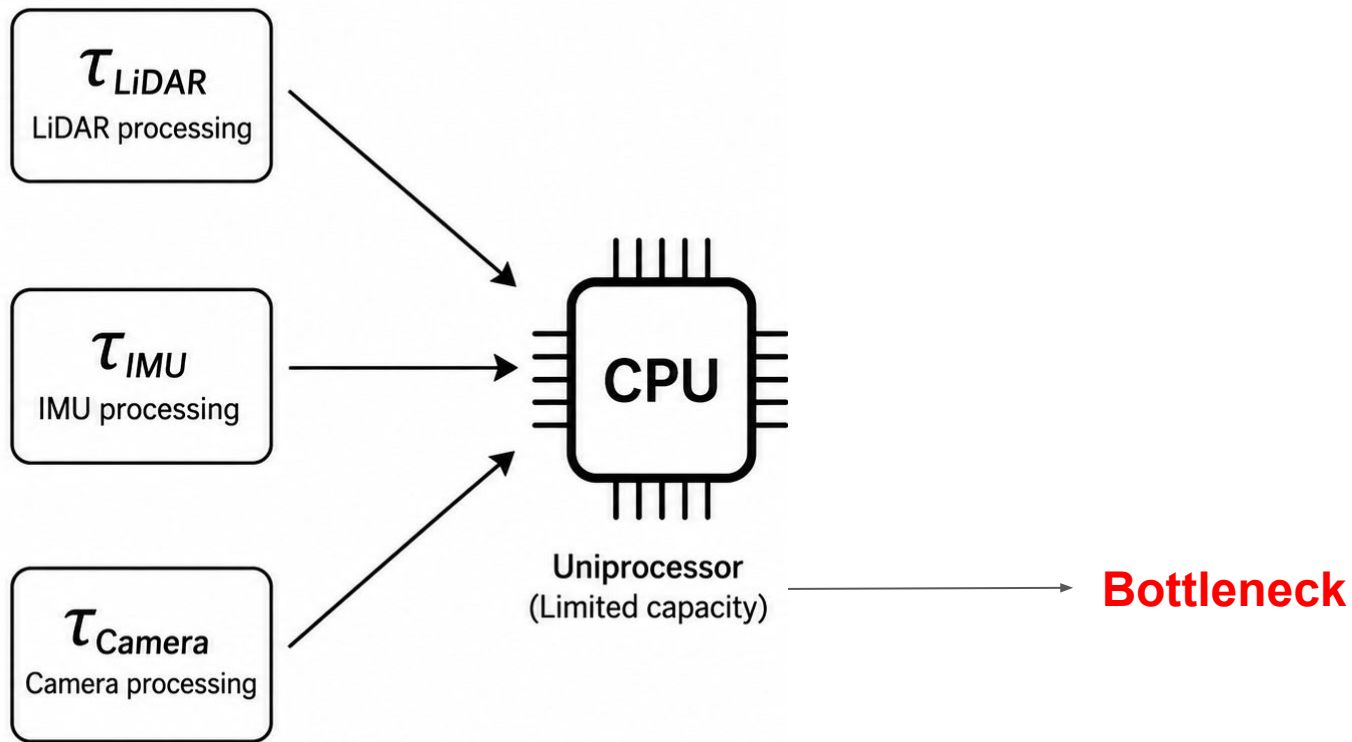
Multiple Sensors



Multiple Sensors



Problem



Need for Scheduling

Set of tasks

Need for Scheduling

Set of tasks  Which task runs first?

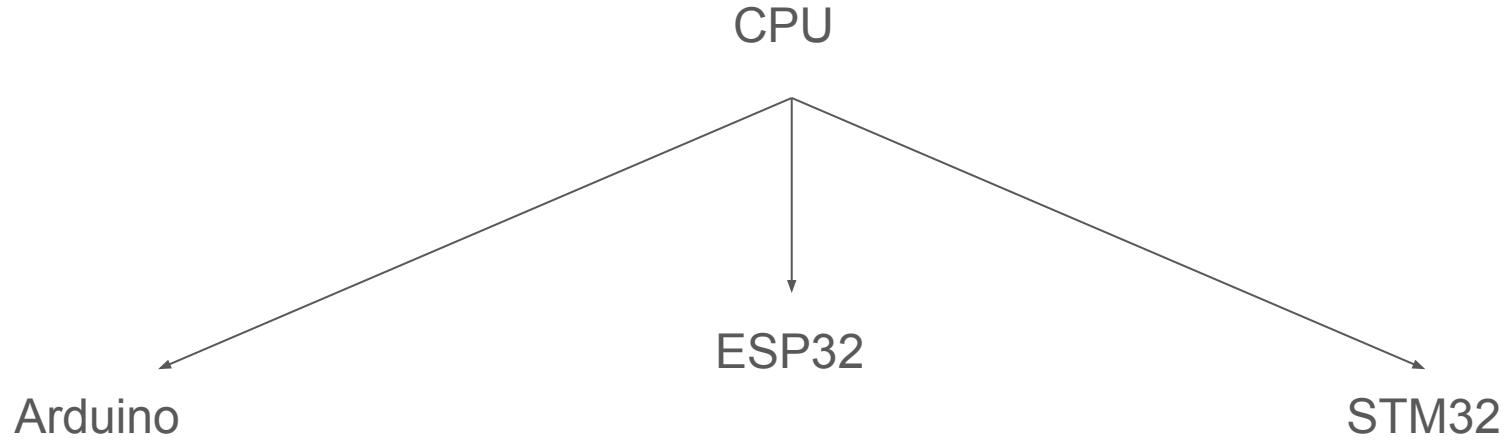
Need for Scheduling

Set of tasks → Which task runs first? → CPU

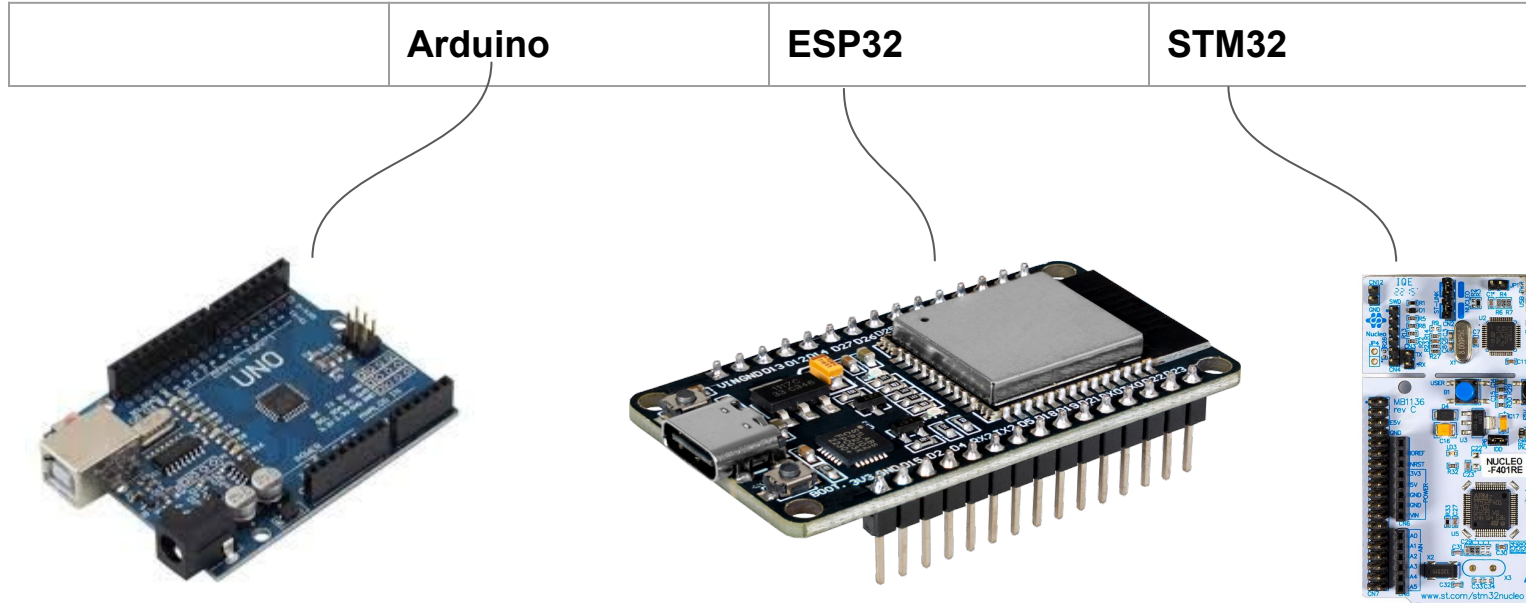
Need for Scheduling

Set of tasks → Which task runs first? → **CPU**

Processor



Processing Units Comparison



Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Minimal setup, ready libraries	Easy, needs config	More setup, low-level code

Processing Units Comparison

	Arduino	ESP32	STM32
<u><i>Ease of use</i></u>	Minimal setup, ready libraries	Easy, needs config	More setup, low-level code

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Minimal setup, ready libraries	Easy, needs config	More setup, low-level code
Performance	Simple tasks	Dual-core, multitasking	High embedded performance

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Minimal setup, ready libraries	Easy, needs config	More setup, low-level code
Performance	Simple tasks	Dual-core, multitasking	High embedded performance
Control	Limited timing control	Medium hardware access	Precise hardware/timing control

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Minimal setup, ready libraries	Easy, needs config	More setup, low-level code
Performance	Simple tasks	Dual-core, multitasking	High embedded performance
Control	Limited timing control	Medium hardware access	Precise hardware/timing control

Why STM32

- Precise control over execution

Why STM32

- Precise control over execution
- Accurate timing measurement

Why STM32

- Precise control over execution
- Accurate timing measurement
- Interrupt-driven behavior

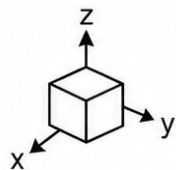
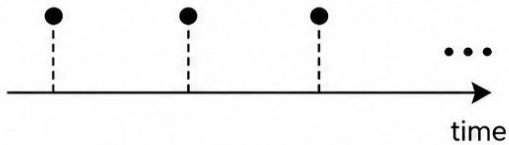
Why STM32

- Precise control over execution
- Accurate timing measurement
- Interrupt-driven behavior
- Low-level access to hardware

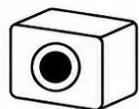
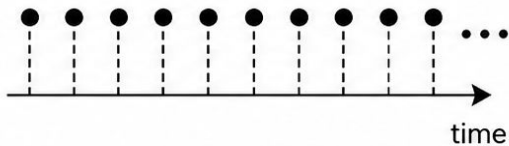
Tasks



LiDAR



IMU



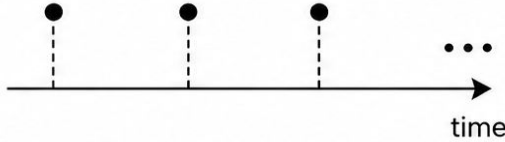
Camera



Tasks



LiDAR

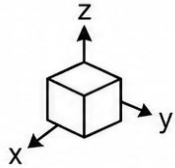


τ_{LiDAR}

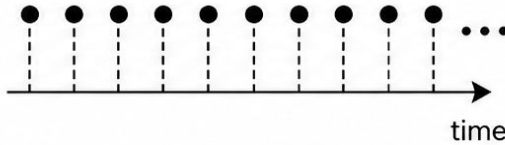
Releases a new job
every T_L

Exec. time: C_L

Deadline: $D_L = T_L$



IMU

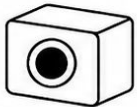


τ_{IMU}

Releases a new job
every T_I

Exec. time: C_I

Deadline: $D_I = T_I$



Camera



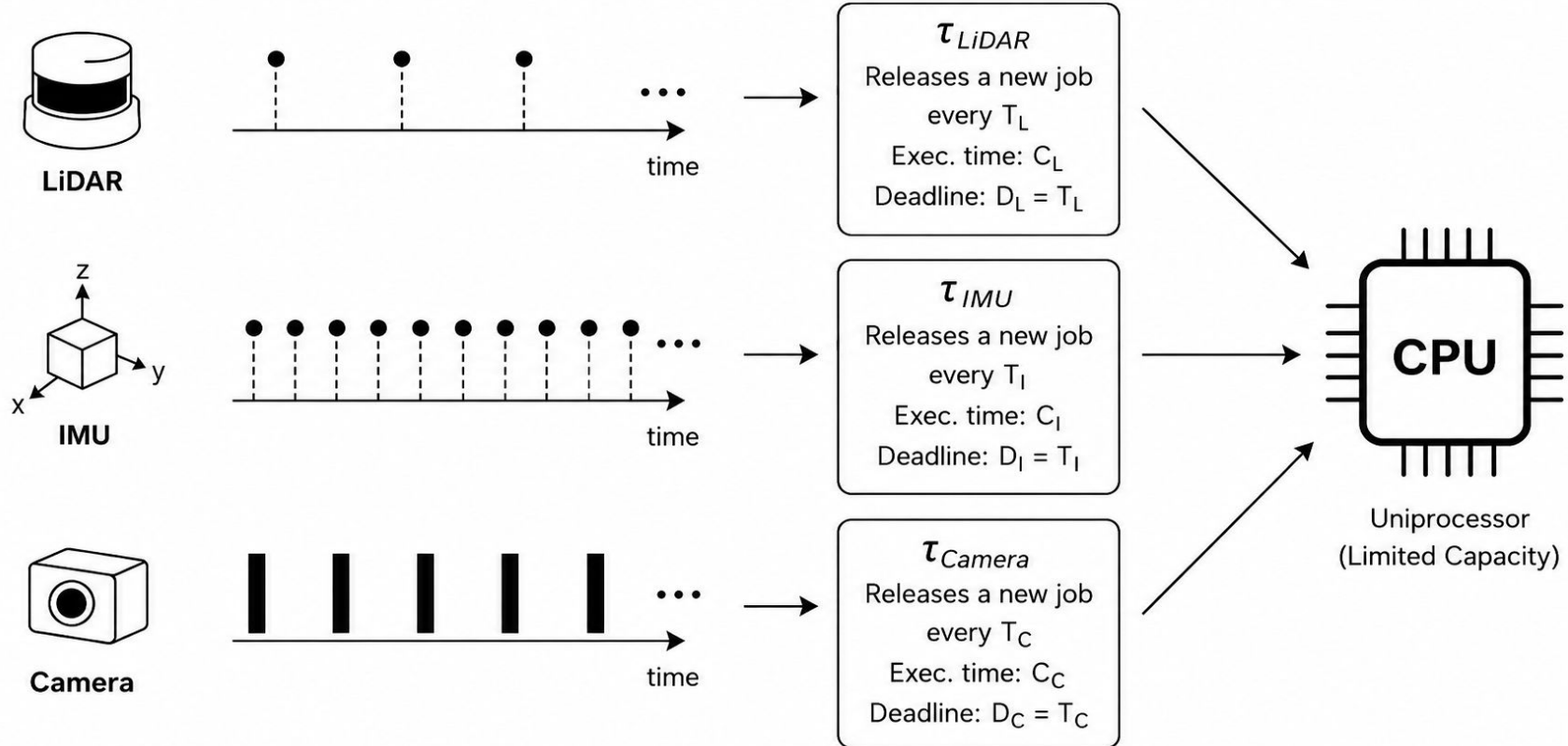
τ_{Camera}

Releases a new job
every T_C

Exec. time: C_C

Deadline: $D_C = T_C$

Tasks

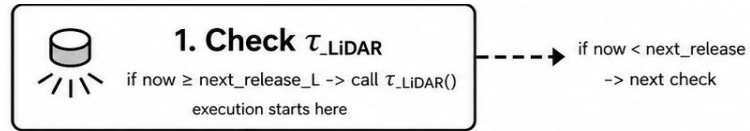


Tasks

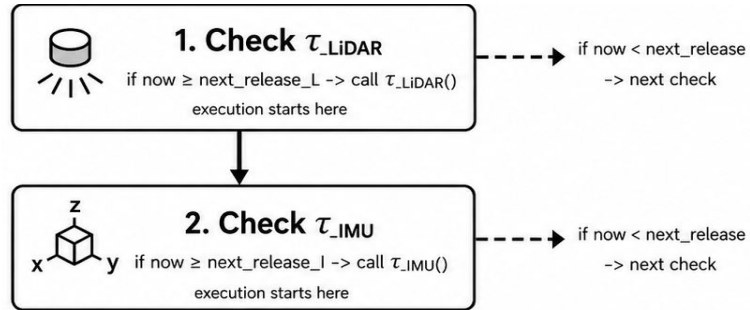
Task Set Summary

Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
τ_{IMU}	Synthetic	Periodic task	$T_I = 10 \text{ ms}$	$C_I = 3 \text{ ms}$	$D_I = T_I = 10 \text{ ms}$	State estimation / stabilization
τ_{LiDAR}	Synthetic	Periodic task	$T_L = 50 \text{ ms}$	$C_L = 20 \text{ ms}$	$D_L = T_L = 50 \text{ ms}$	Environment mapping
τ_{Camera}	Synthetic	Periodic task	$T_C = 200 \text{ ms}$	$C_C = 40 \text{ ms}$	$D_C = T_C = 200 \text{ ms}$	Object detection

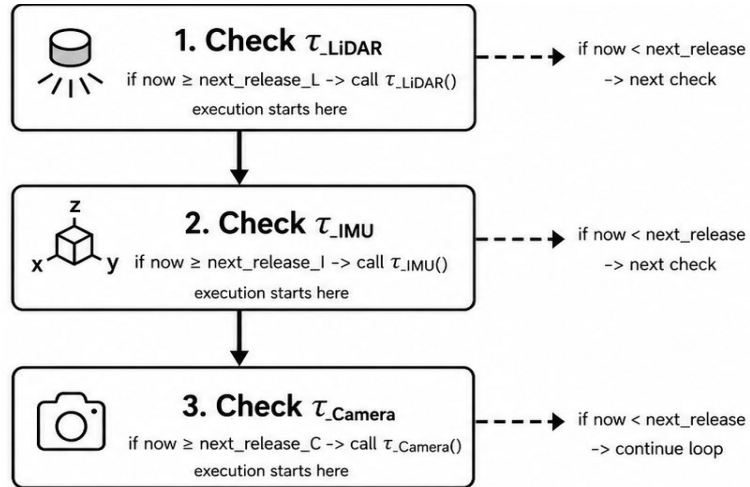
STM32 Execution Logic



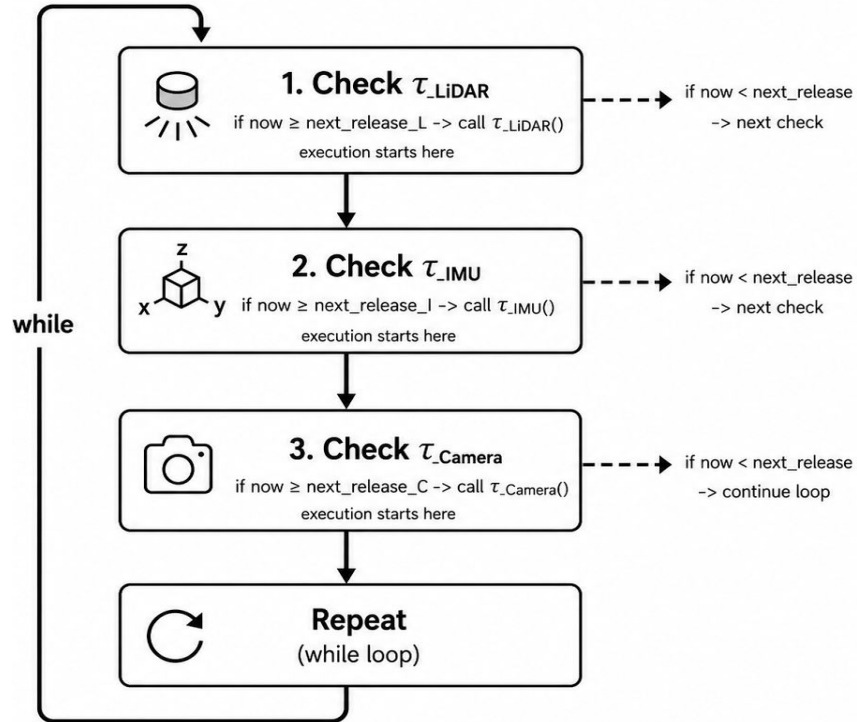
STM32 Execution Logic



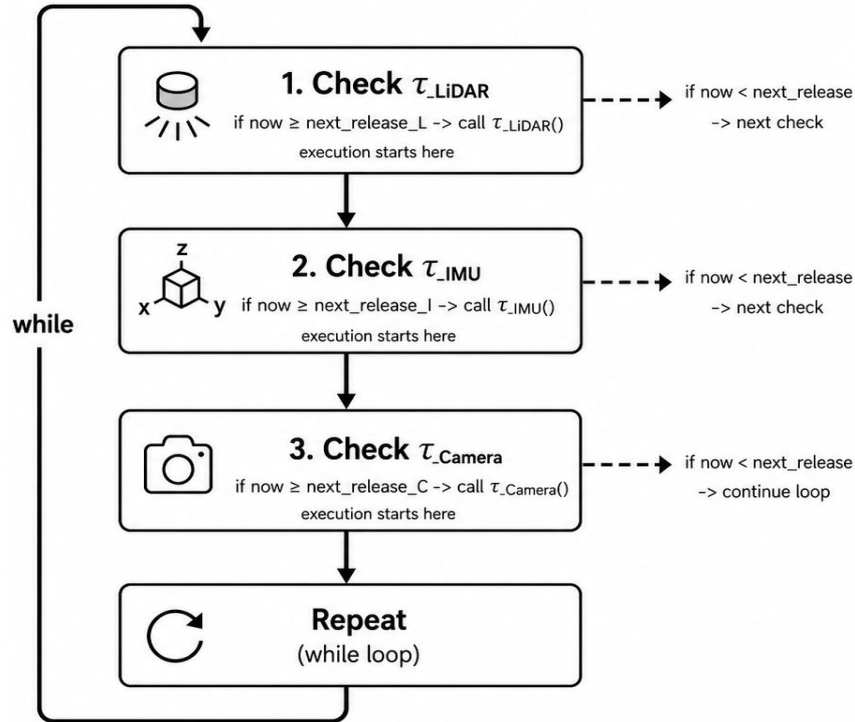
STM32 Execution Logic



STM32 Execution Logic



STM32 Execution Logic



Key Properties

- **Superloop model:**
tasks are checked inside while(1).
- **Sequential task execution:**
 $\tau_{LiDAR} \rightarrow \tau_{IMU} \rightarrow \tau_{Camera}$.
- **Synchronous task activation:**
all tasks generate jobs synchronously at $t = 0$
- **Periodic release:**
each task generates periodic jobs
- **Non-preemptive:**
job execution can not be preempted
- **Uniprocessor execution:**
only one job uses the CPU at a time.
- **Work-conserving execution:**
processor is not IDLE while at least one job is pending
- **No queue:**
no pending-job queue is maintained

If no task is ready, the loop keeps checking until the next release.

Task Formalization

Task model and utilization

$$\tau_i = (C_i, T_i, D_i)$$

$$D_i = T_i$$

C_i - execution time

T_i - period

D_i - relative deadline

$$U_i = \frac{C_i}{T_i}$$

$$U = \sum_i \frac{C_i}{T_i}$$

Task	C	T	U
τ_{IMU}	3 ms	10 ms	0.30
τ_{LiDAR}	20 ms	50 ms	0.40
τ_{Camera}	40 ms	200 ms	0.20
Total $U = 0.90 = 90\%$			

Task Formalization

Task model and utilization

$$\tau_i = (C_i, T_i, D_i)$$

$$D_i = T_i$$

C_i - execution time

T_i - period

D_i - relative deadline

$$U_i = \frac{C_i}{T_i}$$

$$U = \sum_i \frac{C_i}{T_i}$$

Task	C	T	U
τ_{IMU}	3 ms	10 ms	0.30
τ_{LiDAR}	20 ms	50 ms	0.40
τ_{Camera}	40 ms	200 ms	0.20
Total $U = 0.90 = 90\%$			

Timing metrics

1 Response time: $R_i^k = f_i^k - r_i^k$
time from release to completion

2 Deadline: $d_i^k = r_i^k + D_i$
absolute deadline of the k -th job

3 Deadline miss if: $f_i^k > d_i^k$
equivalently: $R_i^k > D_i$

4 Lateness: $L_i^k = f_i^k - d_i^k = R_i^k - D_i$
positive value means late completion

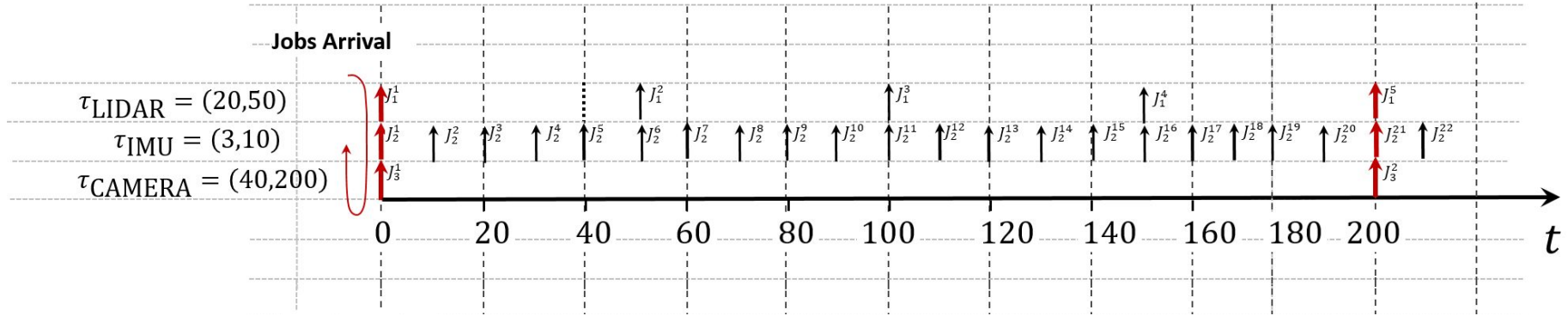
5 Skipped releases: $skipped_i = expected\ releases_i - handled\ jobs_i$
releases not stored in the no-queue superloop

6 Total timing failures: $failures_i = deadline\ misses_i + skipped\ releases_i$

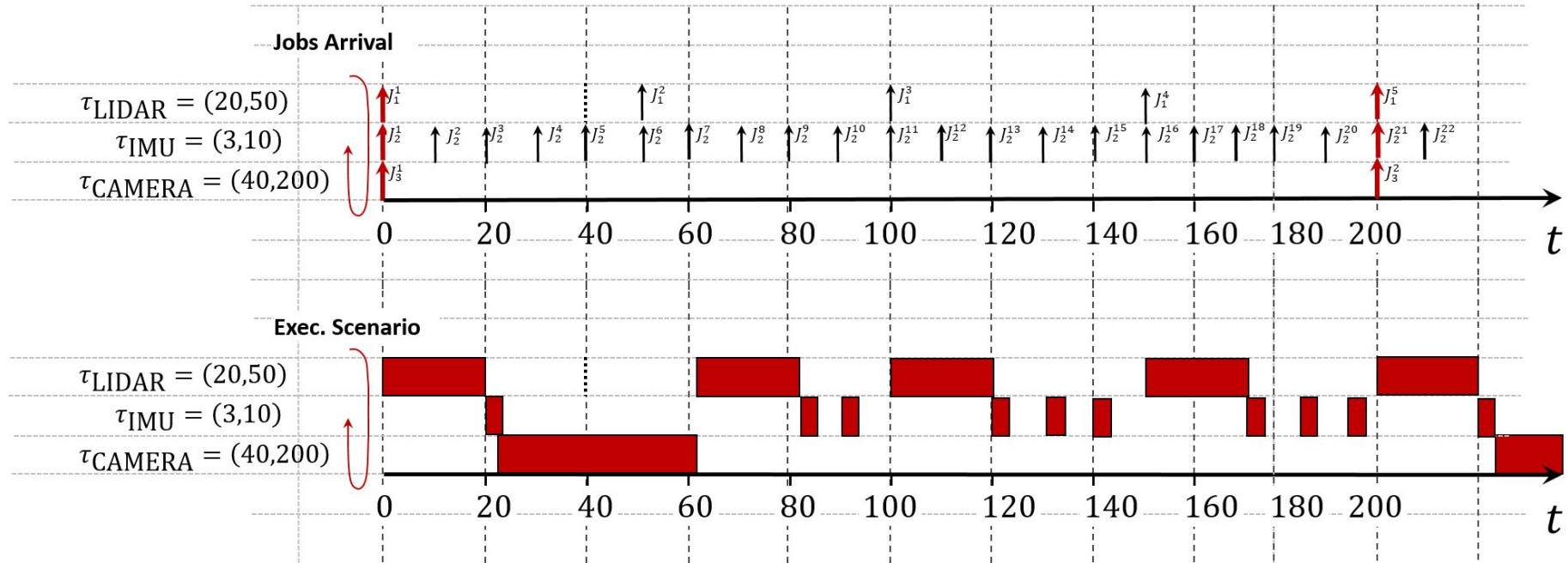
r_i^k - release time of the k -th job

f_i^k - finish time of the k -th job

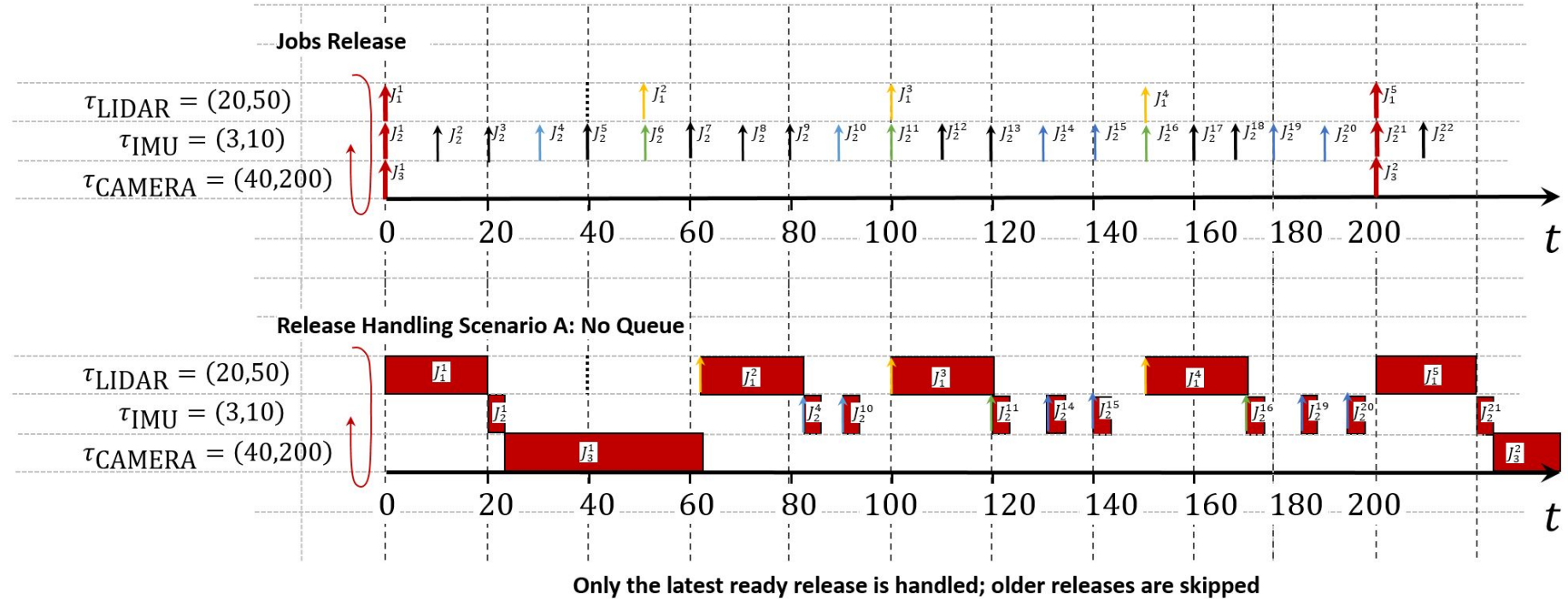
CPU Execution Timeline



CPU Execution Timeline



CPU Execution Timeline



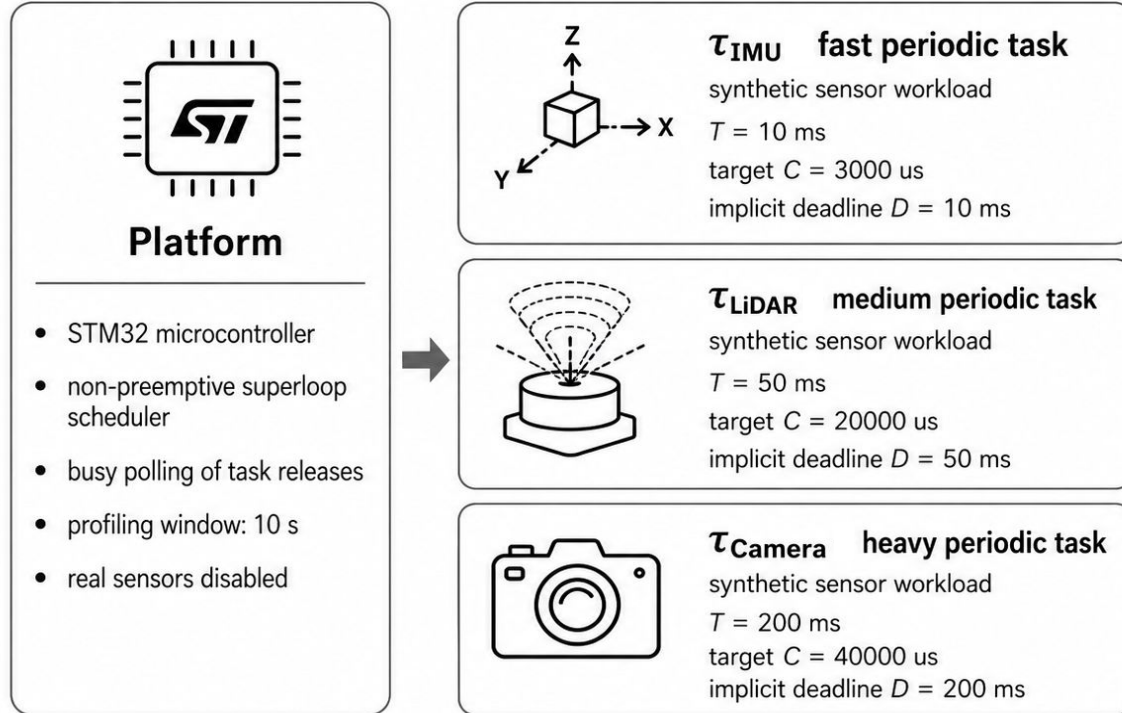
Profiling: Cost of Sequential Checks



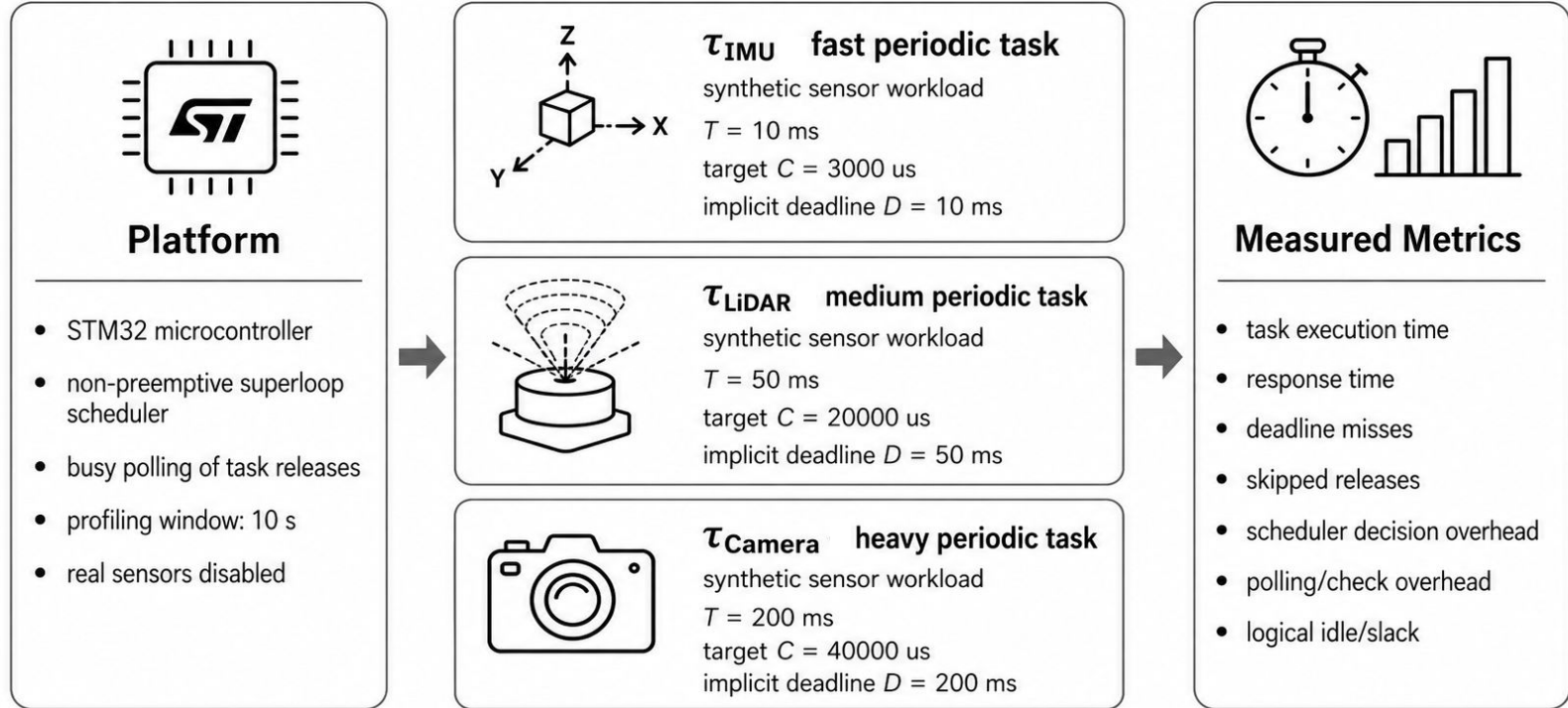
Platform

- STM32 microcontroller
- non-preemptive superloop scheduler
- busy polling of task releases
- profiling window: 10 s
- real sensors disabled

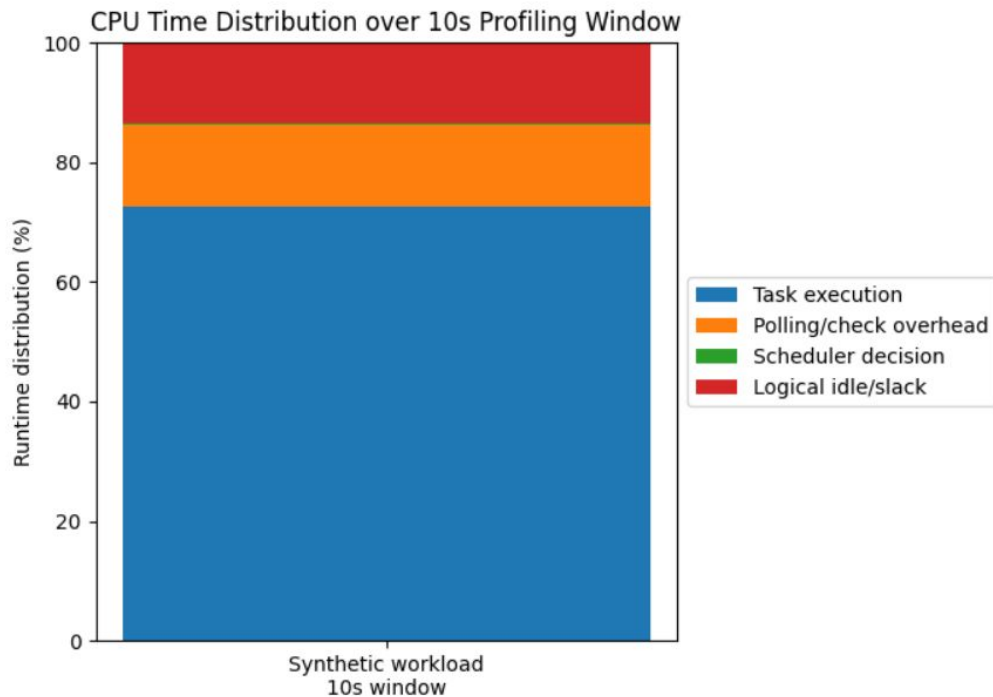
Profiling: Cost of Sequential Checks



Profiling: Cost of Sequential Checks



Profiling: Results



CPU Time Distribution Summary

Metric	Value (%)
task_exec_percent	72.6901
polling_check_percent	13.762
scheduler_decision_percent	0.0612
logical_idle_percent	13.5477
measured_busy_percent	86.5133
sum	100.061

Profiling: Failures and Response Statistics

Task	Runs	Response avg (us)	Response min (us)	Response max (us)	Misses	Max lateness (us)	Skipped releases	Total timing failures
------	------	----------------------	----------------------	----------------------	--------	----------------------	---------------------	--------------------------

Profiling: Failures and Response Statistics

Task	Runs	Response avg (us)	Response min (us)	Response max (us)	Misses	Max lateness (us)	Skipped releases	Total timing failures
tau_IMU synthetic	454	15387	3032	83142	189	73142	548	737

Profiling: Failures and Response Statistics

Task	Runs	Response avg (us)	Response min (us)	Response max (us)	Misses	Max lateness (us)	Skipped releases	Total timing failures
tau_IMU synthetic	454	15387	3032	83142	189	73142	548	737
tau_LiDAR synthetic	195	23931	20027	60082	5	10082	5	10

Profiling: Failures and Response Statistics

Task	Runs	Response avg (us)	Response min (us)	Response max (us)	Misses	Max lateness (us)	Skipped releases	Total timing failures
tau_IMU synthetic	454	15387	3032	83142	189	73142	548	737
tau_LiDAR synthetic	195	23931	20027	60082	5	10082	5	10
tau_Camera synthetic	50	60423	40024	63104	0	0	0	0

Profiling: Failures and Response Statistics

Task	Runs	Response avg (us)	Response min (us)	Response max (us)	Misses	Max lateness (us)	Skipped releases	Total timing failures
tau_IMU synthetic	454	15387	3032	83142	189	73142	548	737
tau_LiDAR synthetic	195	23931	20027	60082	5	10082	5	10
tau_Camera synthetic	50	60423	40024	63104	0	0	0	0

Problem

Why is this a problem?

- High-frequency tasks are delayed

Why is this a problem?

- High-frequency tasks are delayed
- Long tasks block execution

Why is this a problem?

- High-frequency tasks are delayed
- Long tasks block execution
- Timing is limited by code order

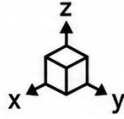
Scheduling Problem



LiDAR

$T = 50 \text{ ms}$

$C = 20 \text{ ms}$



IMU

$T = 10 \text{ ms}$

$C = 3 \text{ ms}$

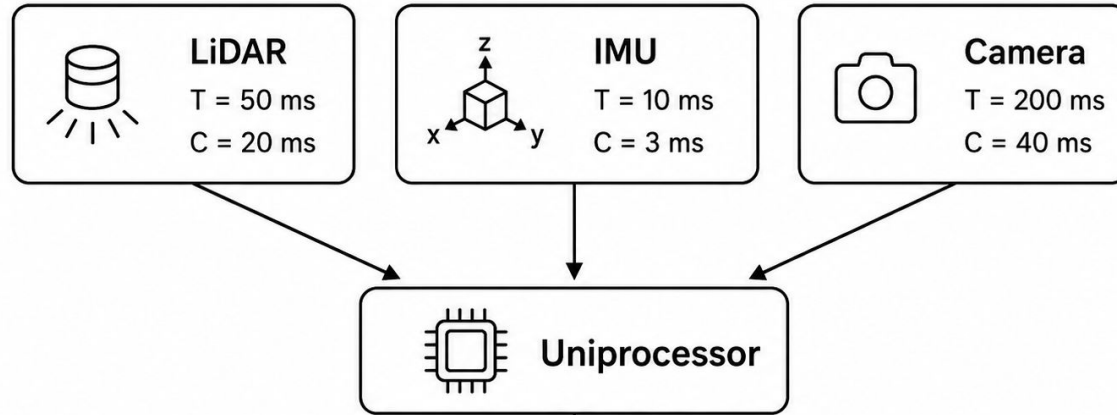


Camera

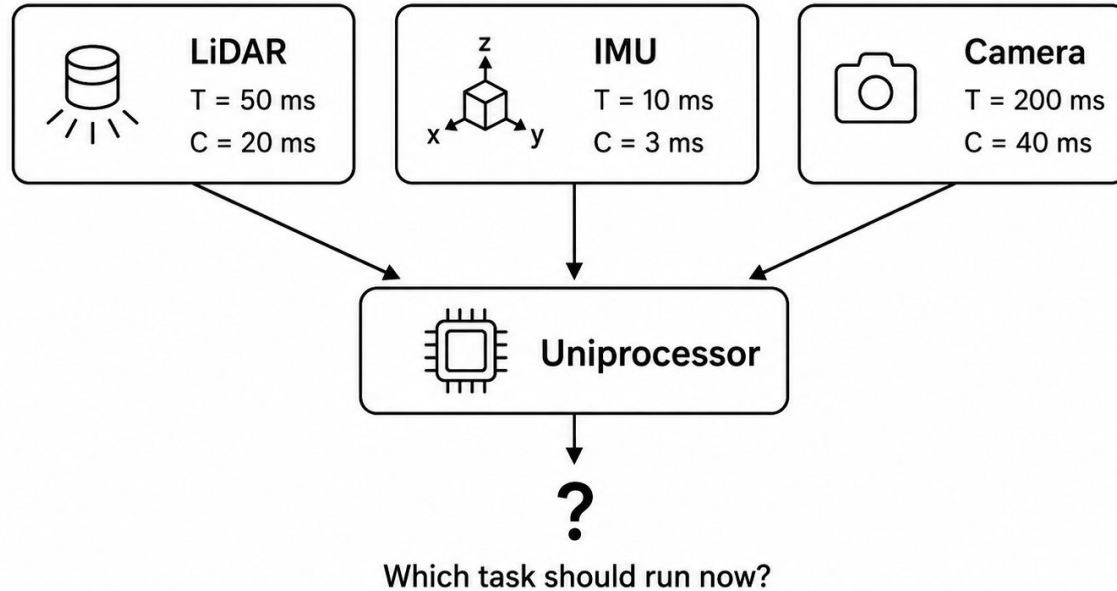
$T = 200 \text{ ms}$

$C = 40 \text{ ms}$

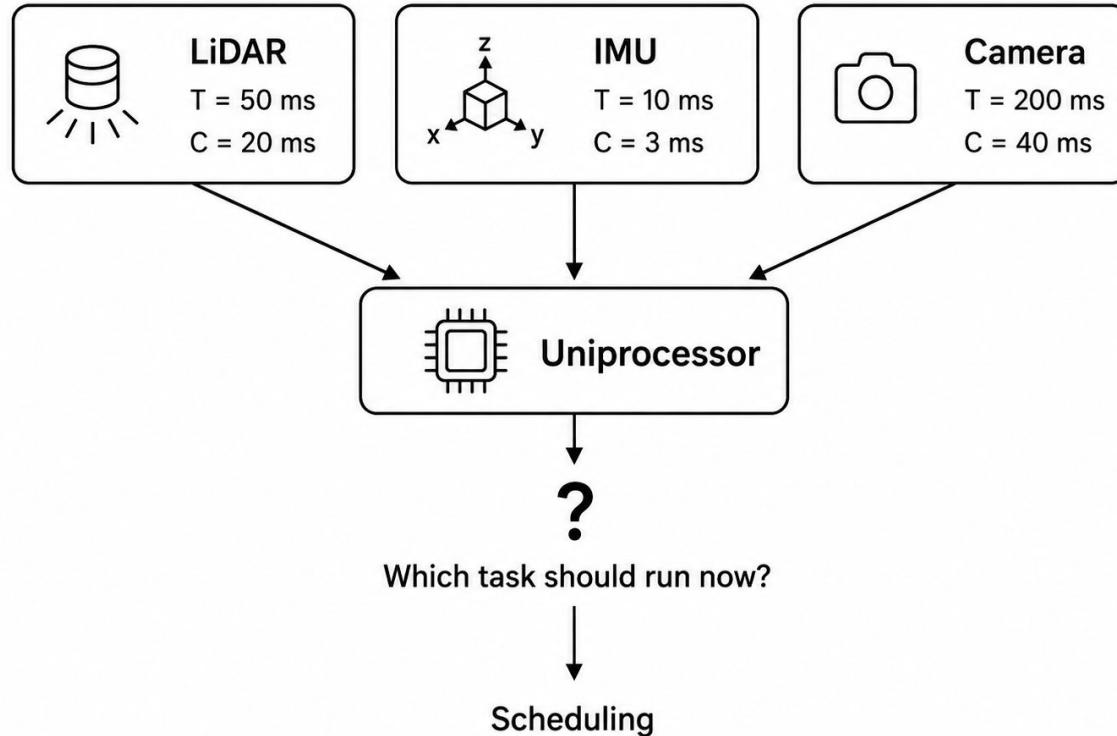
Scheduling Problem



Scheduling Problem



Scheduling Problem



Earliest Deadline First (EDF)

Task	Period	Relative deadline
------	--------	-------------------

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	10 ms	10 ms

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	10 ms	10 ms
LiDAR	50 ms	50 ms

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	10 ms	10 ms
LiDAR	50 ms	50 ms
Camera	200 ms	200 ms

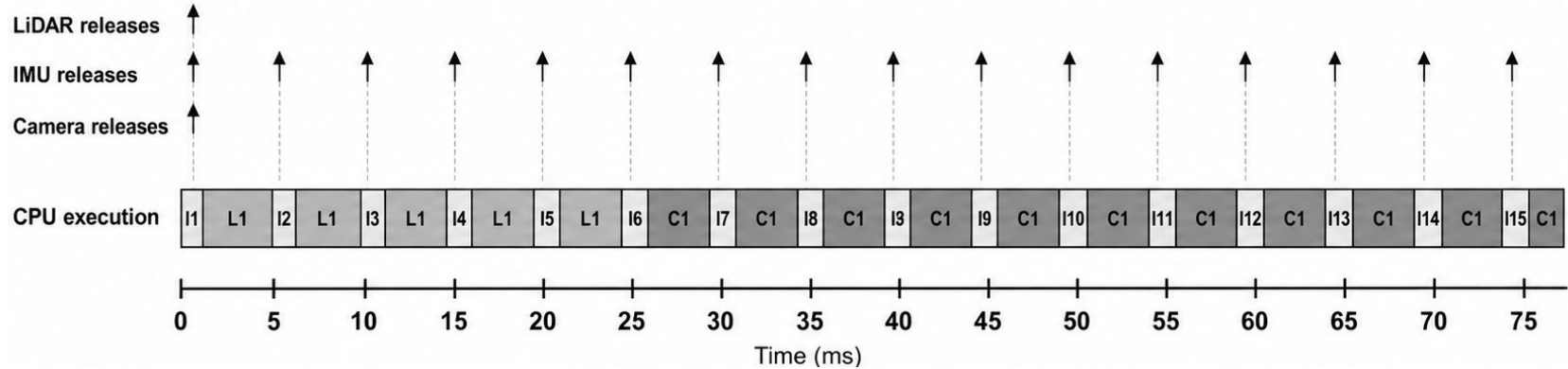
Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	10 ms	10 ms
LiDAR	50 ms	50 ms
Camera	200 ms	200 ms

EDF Execution Timeline

Task Set Summary

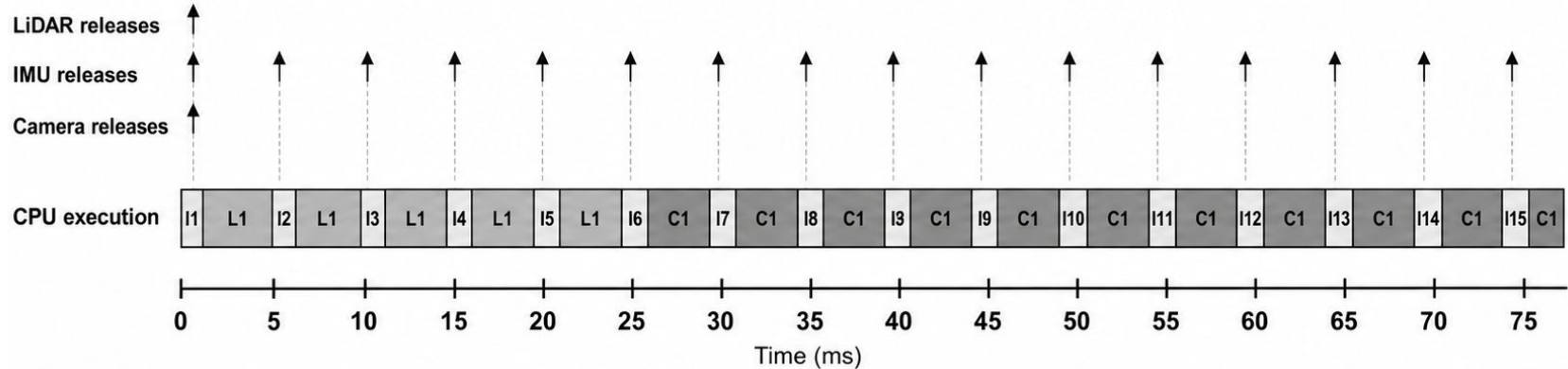
Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	$T_L = 100$ ms	$C_L = 20$ ms	$D_L = T_L = 100$ ms	Environment mapping
Task 2	IMU	Periodic (High Rate)	$T_I = 5$ ms	$C_I = 1$ ms	$D_I = T_I = 5$ ms	State estimation / stabilization
Task 3	Camera	Periodic	$T_C = 200$ ms	$C_C = 40$ ms	$D_C = T_C = 200$ ms	Object detection



EDF Execution Timeline

Task Set Summary

Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	$T_L = 100$ ms	$C_L = 20$ ms	$D_L = T_L = 100$ ms	Environment mapping
Task 2	IMU	Periodic (High Rate)	$T_I = 5$ ms	$C_I = 1$ ms	$D_I = T_I = 5$ ms	State estimation / stabilization
Task 3	Camera	Periodic	$T_C = 200$ ms	$C_C = 40$ ms	$D_C = T_C = 200$ ms	Object detection



- EDF always runs the ready job with the earliest absolute deadline.
- IMU preempts LiDAR and Camera every 5 ms because each IMU job has the closest deadline.
- LiDAR (L1) completes at 25 ms after receiving 20 ms total execution.
- Camera (C1) starts at 26 ms and completes at 75 ms after 40 ms total execution.

Limitation of STM32 Execution Model

Aspect	EDF	STM32
---------------	------------	--------------

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion
Timing guarantees	Strong guarantees	Hard to guarantee

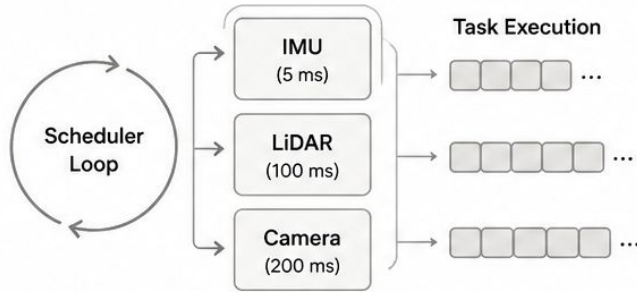
Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion
Timing guarantees	Strong guarantees	Hard to guarantee

Design Choice: Implementing Preemption on STM32

1. Task Splitting (Chunks)

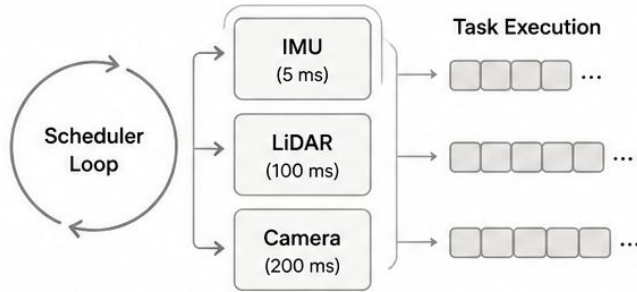
Break tasks into small execution steps
and schedule cooperatively



Design Choice: Implementing Preemption on STM32

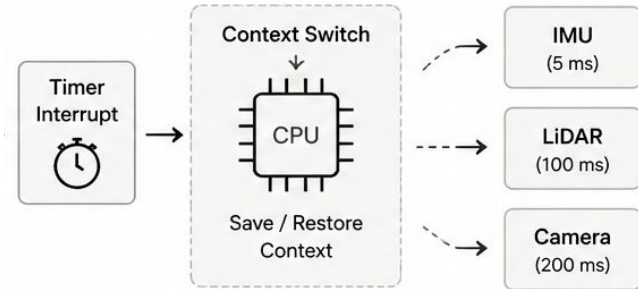
1. Task Splitting (Chunks)

Break tasks into small execution steps
and schedule cooperatively

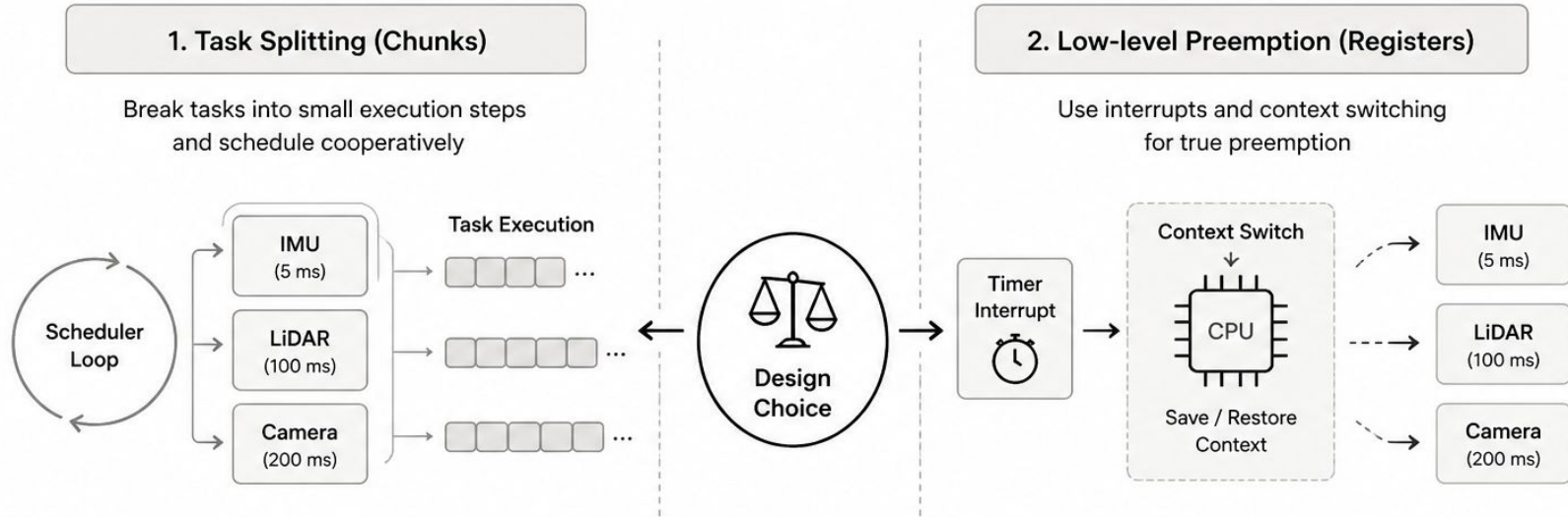


2. Low-level Preemption (Registers)

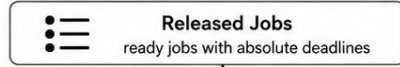
Use interrupts and context switching
for true preemption



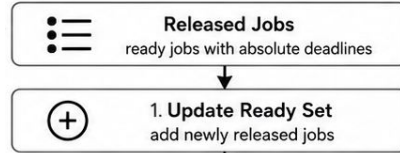
Design Choice: Implementing Preemption on STM32



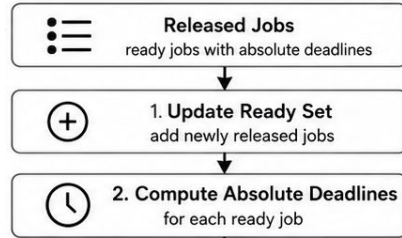
STM32 Execution Logic (EDF)



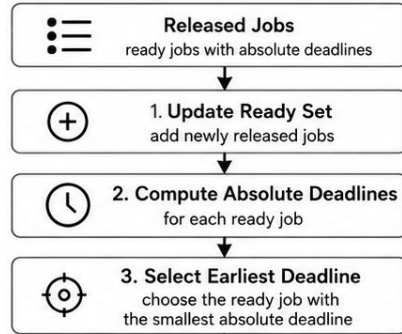
STM32 Execution Logic (EDF)



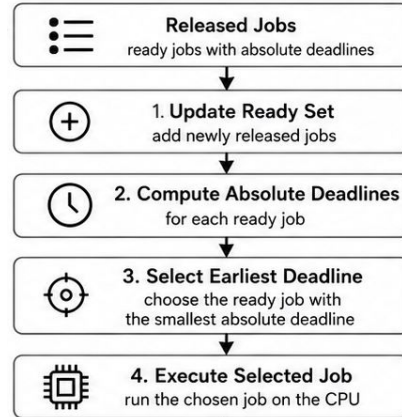
STM32 Execution Logic (EDF)



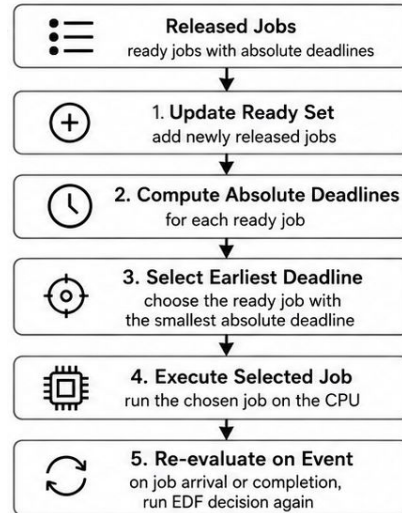
STM32 Execution Logic (EDF)



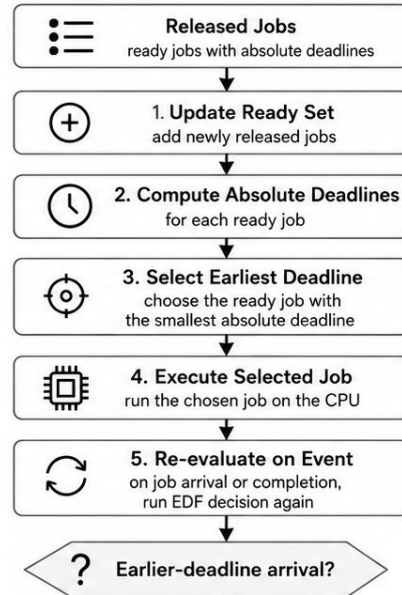
STM32 Execution Logic (EDF)



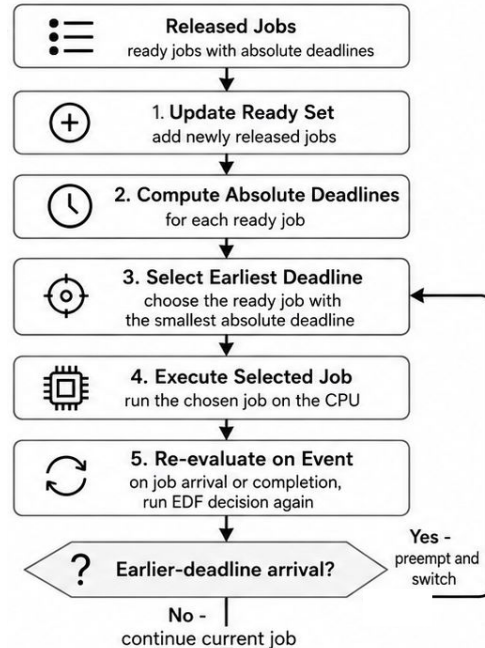
STM32 Execution Logic (EDF)



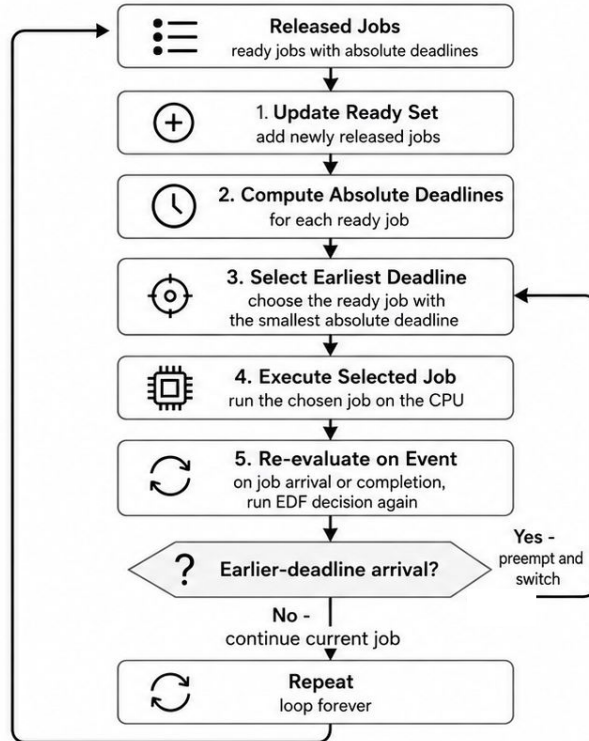
STM32 Execution Logic (EDF)



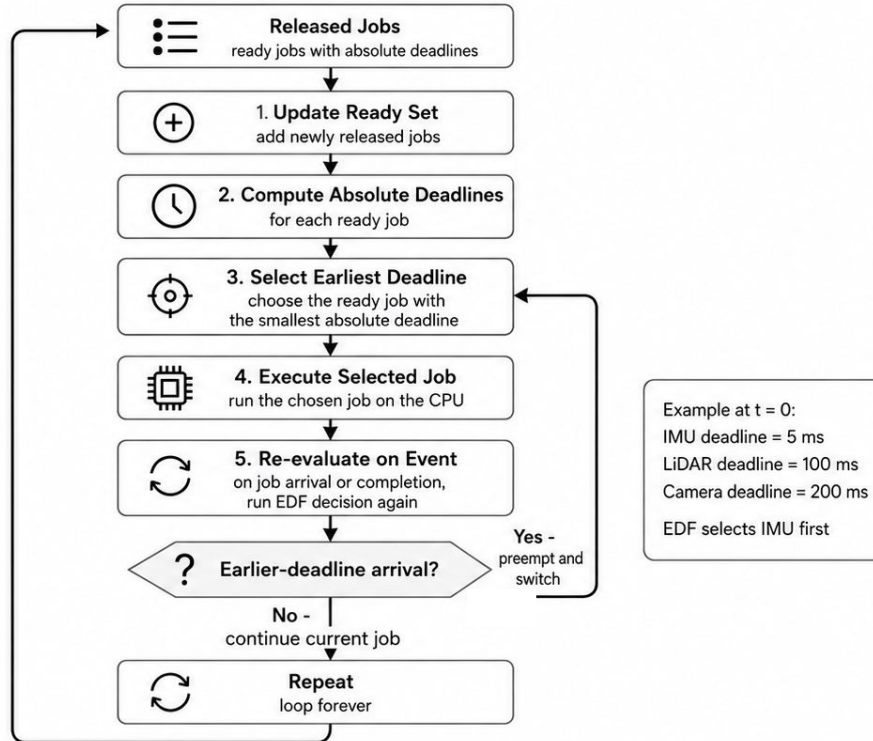
STM32 Execution Logic (EDF)



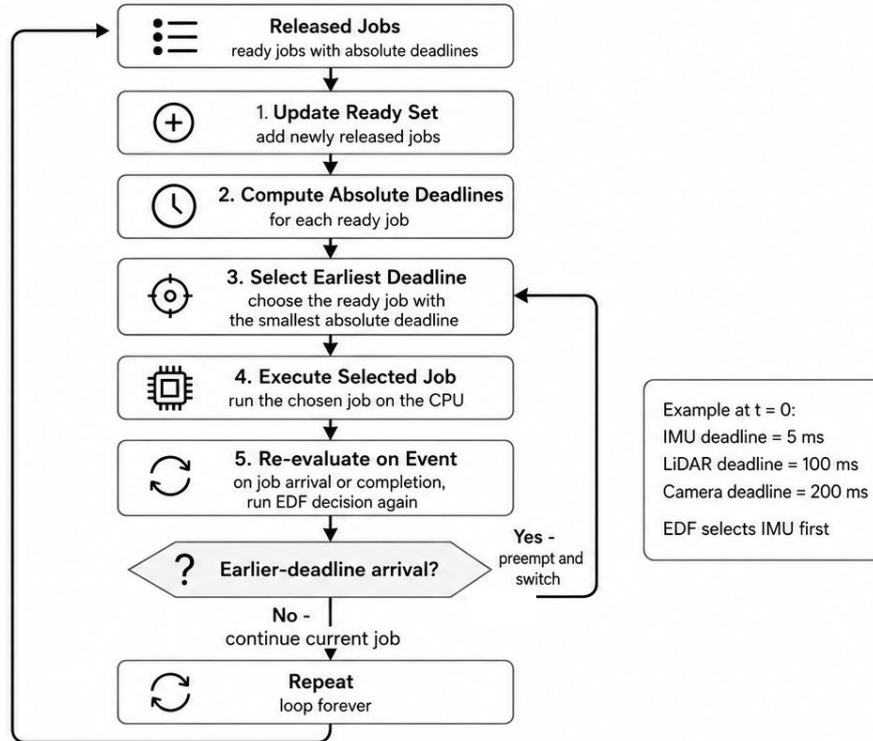
STM32 Execution Logic (EDF)



STM32 Execution Logic (EDF)



STM32 Execution Logic (EDF)



Example at $t = 0$:
IMU deadline = 5 ms
LiDAR deadline = 100 ms
Camera deadline = 200 ms
EDF selects IMU first

Key Properties

- **Dynamic priority:**
priority changes with absolute deadlines
- **EDF rule:**
earliest absolute deadline runs first
- **Preemptive model:**
a newly arrived earlier-deadline job can interrupt the current one
- **One task at a time:**
only one job uses the CPU at a given moment
- **Deadline-based scheduling:**
order is decided by deadlines, not fixed code order

Task Output

Sensor	What we measure	Processed data	Used for
--------	-----------------	----------------	----------

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance
IMU	Orientation / motion	Angle, rotation	Stabilization

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance
IMU	Orientation / motion	Angle, rotation	Stabilization
Camera	Images	Detected objects	Navigation / tracking

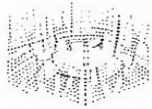
Decision Making

1. Sensor Inputs



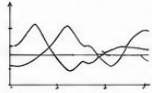
LiDAR

Measures distance
to surroundings



IMU

Measures motion
and orientation

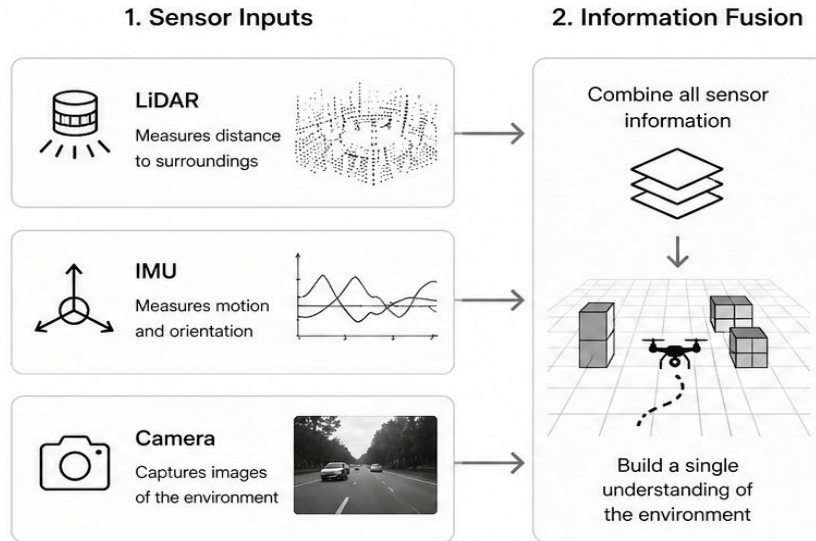


Camera

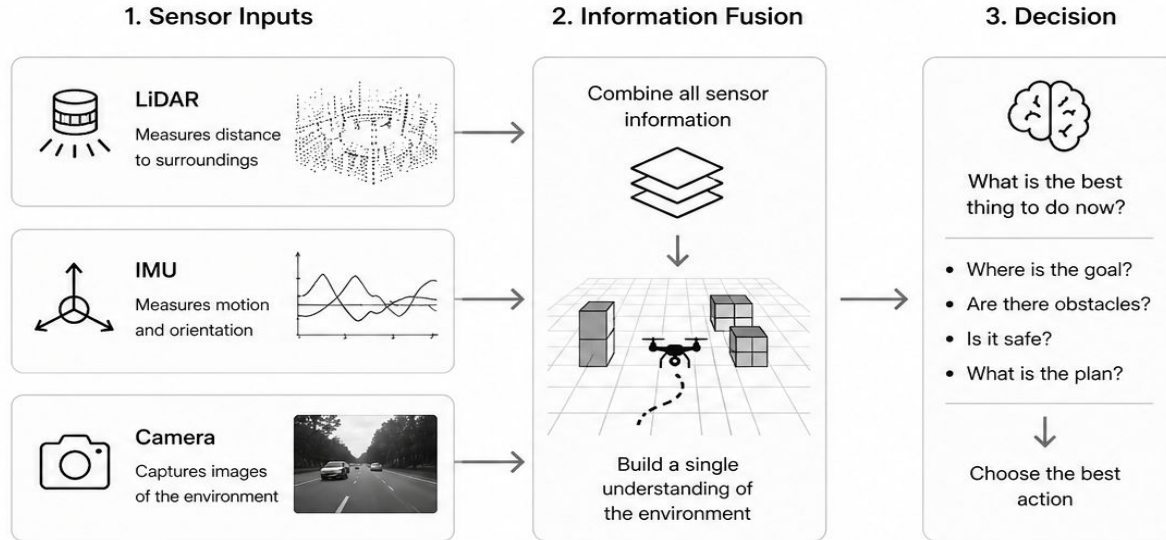
Captures images
of the environment



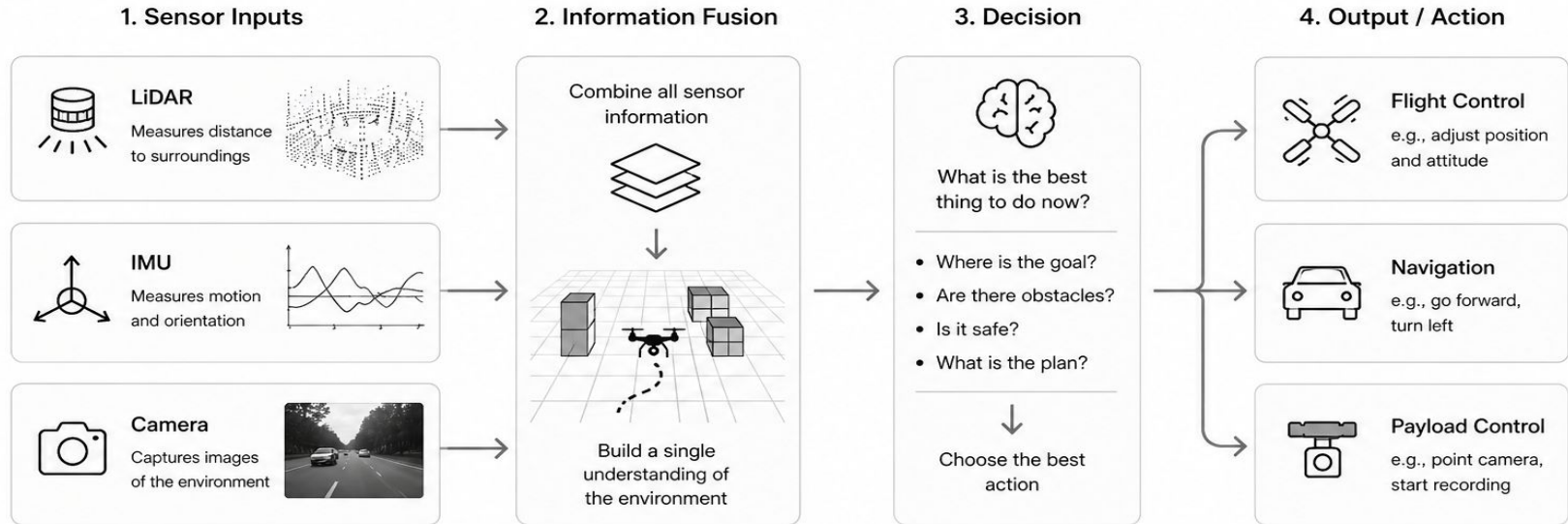
Decision Making



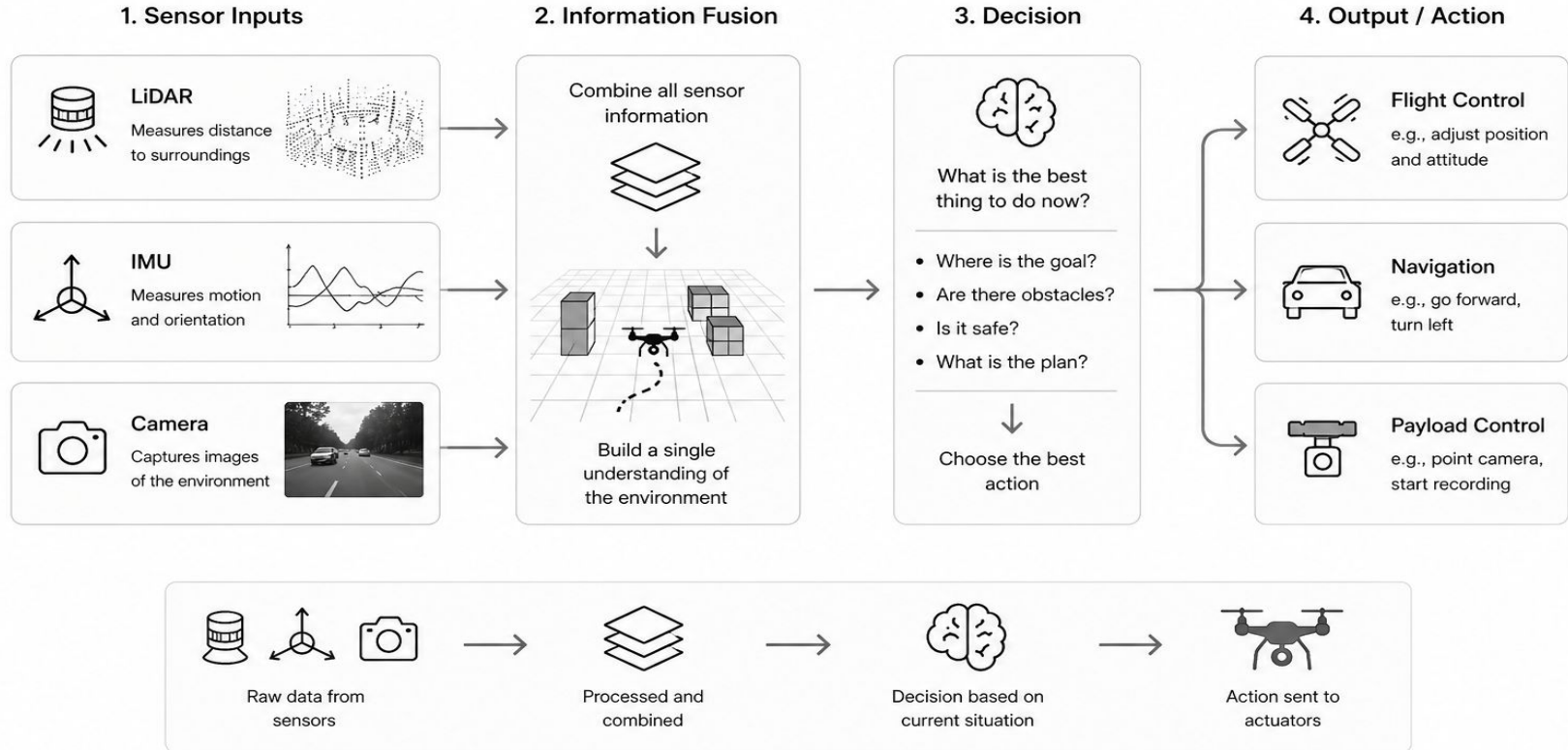
Decision Making



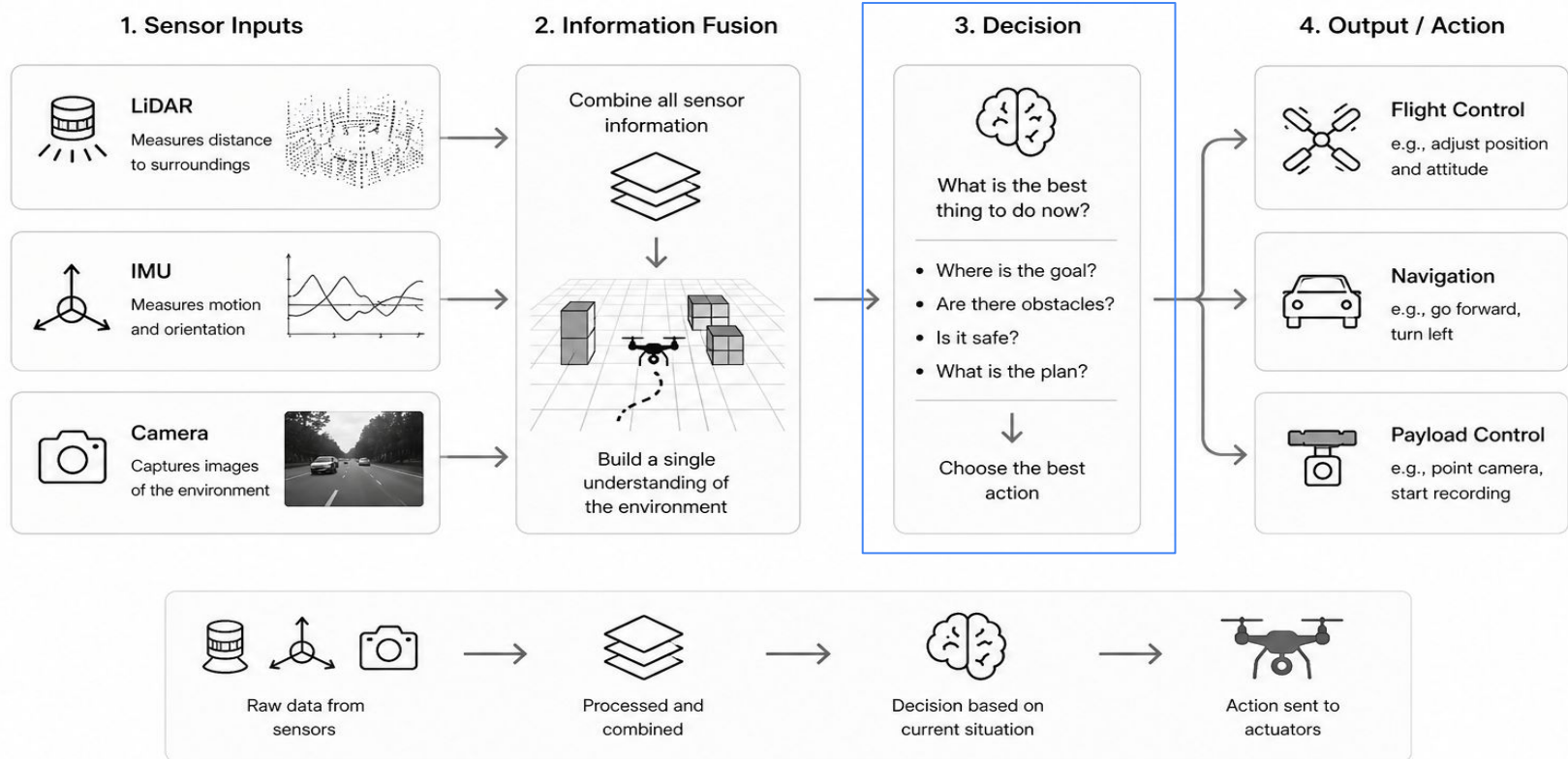
Decision Making



Decision Making



Decision Making



Control Commands

1. Decision



Based on the current
understanding of the environment.

Examples:



Obstacle ahead
→ Avoid it



Strong wind / Disturbance
→ Stabilize



Target detected
→ Track it

Control Commands

1. Decision



Based on the current understanding of the environment.

Examples:



Obstacle ahead
→ Avoid it



Strong wind / Disturbance
→ Stabilize



Target detected
→ Track it



2. Control Commands

The decision is converted into specific control commands.



Turn left
→ Change direction



Increase thrust
→ Gain speed / altitude

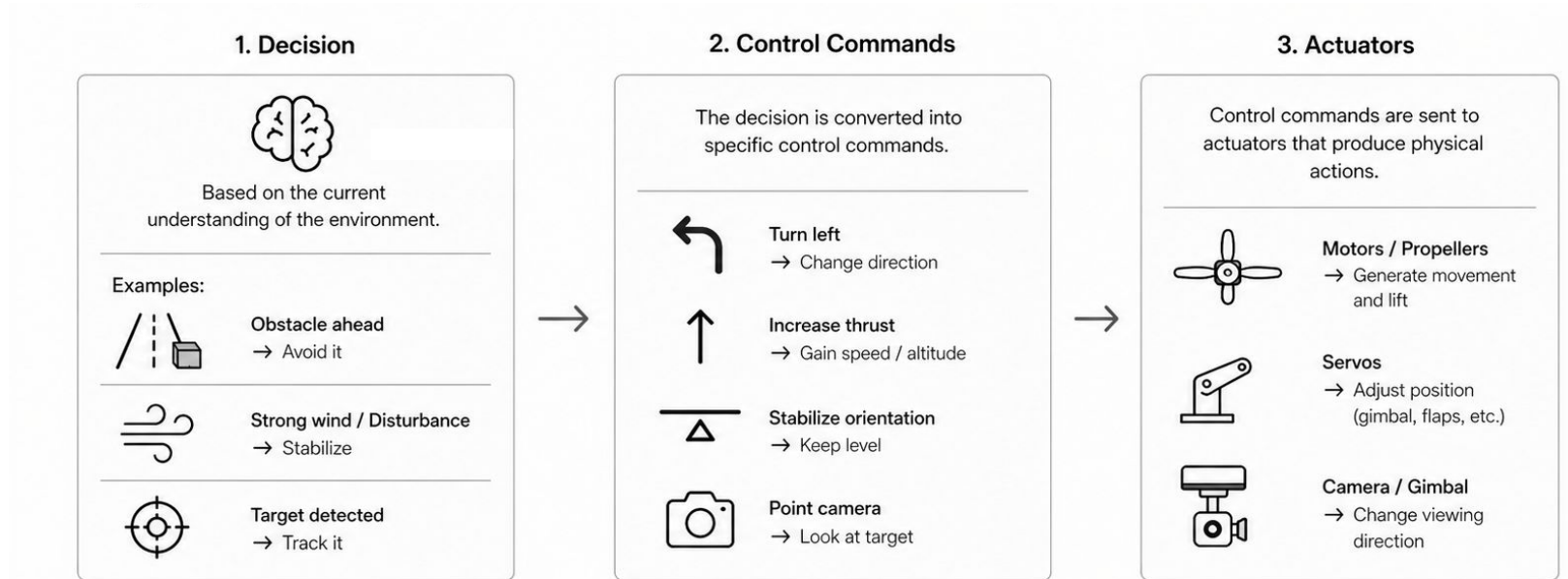


Stabilize orientation
→ Keep level



Point camera
→ Look at target

Actuators



Closed Loop System

